

Breaking the Barrier for Asynchronous MPC with a Friend

Banashri Karmakar¹, Aniket Kate², Shravani Patil¹, Arpita Patra¹, Sikhar Patranabis³, Protik Paul⁴, and Divya Ravi⁵

¹ Indian Institute of Science, Bangalore, Emails: {banashrik, arpita}@iisc.ac.in, patilshravani95@gmail.com

² Supra Research / Purdue University, Email: aniket@purdue.edu

³ IBM Research India, Email: sikhar.patranabis@ibm.com

⁴ Technische Universität Darmstadt, Email: protik.pmax.paul@gmail.com

⁵ University of Amsterdam, Netherlands, Email: d.ravi@uva.nl

Abstract. Multiparty computation (MPC) is a topic of growing interest for privacy-preserving computation tasks. A few MPC libraries have been developed, and newer protocols are regularly proposed to reduce the latency overhead, improve scalability, and achieve strong termination guarantees. However, most current MPC protocols are designed and implemented assuming network synchrony: in theory, they assume that all messages are delivered within a known time bound, while for experimental analysis, most assume all nodes to be honest, such that the time bounds are never deployed. While deploying MPC systems in the wild and trying to minimize the latency, network synchrony is indeed a strong assumption to make: natural adverse network conditions can break the safety and/or liveness of the protocol due to simply delayed messages.

Asynchronous MPC (AMPC) protocols can overcome the challenge as they do not assume fixed time bounds for message delivery delays; however, AMPC faces a natural threshold barrier of 2/3rd honest majority and introduces significant computation and/or communication overheads. This work aims to achieve the best-of-both network models by designing a practical AMPC protocol that has stronger resilience guarantees matching those for synchronous MPC.

We achieve this by adopting the emerging *helper-aided model*, and designing protocols that achieve *fairness* not only in the simple honest majority setting but also in the dishonest majority setting. Our protocols follow the standard *preprocessing-online* paradigm, enabling a lightweight and fast input-dependent online phase. In the honest majority setting, our protocol relies solely on lightweight cryptographic operations. In the dishonest majority setting, the protocol requires *oblivious transfer (OT)* during preprocessing, which we prove is necessary in this setting. We implement our constructions and provide a thorough performance comparison with state-of-the-art MPC protocols in the helper-aided model. Our experiments demonstrate that our protocols substantially outperform the state-of-the-art helper-aided MPC scheme, while being significantly more resilient to network delays.

1 Introduction

Multi-Party Computation (MPC) [42] allows multiple parties to jointly and securely compute a function of their private data. Practically deployed MPC solutions often need to operate over real-world networks, such as the Internet, which inevitably suffer from arbitrary delays and message dropouts. This makes MPC protocols designed for *asynchronous* networks more suitable for real-world applications. However, the vast majority of the MPC literature focuses on the more ideal *synchronous* setting, which assumes that all participants have access to a synchronized global clock and all messages are delivered within a pre-determined timeout.

Synchronous MPC protocols naturally provide stronger security guarantees and are also simpler to design as compared to their asynchronous counterparts. For instance, synchronous MPC protocols in the computational setting can tolerate up to $t < n$ active/malicious corruptions while providing abort security [21], while asynchronous MPC (AMPC) protocols have a much lower resilience of $t < n/3$ malicious corruptions for the same setting and comparable notions of security [39] (recalled in Appendix A). The main challenge in the asynchronous setting is that an honest party that does not receive an expected message cannot distinguish between an honest-but-slow sender (whose message got delayed due to network issues) and a maliciously corrupt sender (that did not send a message at all). However, in synchronous MPC, an honest party that

does not receive an expected message within a pre-determined time bound simply concludes that the sender is maliciously corrupt. Furthermore, AMPC protocols are inherently unable to guarantee strong termination; in the presence of even a single crashed party, there exists at least one execution in which the protocol fails to terminate [22].

In this paper, we ask whether one could design practically efficient MPC protocols that combine the stronger security guarantees (equivalently, higher active corruption thresholds and strong termination guarantees) of synchronous MPC protocols with the ability to operate asynchronously. To the best of our knowledge, such an MPC protocol does not exist. A prior work [4] attempted to bridge the gap between synchronous and asynchronous MPC by designing an AMPC protocol that achieves full security while tolerating up to $t < n/2$ parties that are either honest-but-slow or maliciously corrupt. However, this construction is far from practical as it relies on computationally expensive cryptographic primitives such as threshold homomorphic encryption, requires each party to prove complex statements in zero-knowledge, and also crucially relies on strong (trusted) setup assumptions.

Helper-Aided MPC. A recent line of works [27,29] has studied new model of synchronous MPC, called *helper-aided MPC*. This model assumes an additional, passively corrupt (i.e., semi-honest) party that acts as a designated *helper party* (abbreviated as HP). The HP may provide an input, but typically does not receive any outputs, and mainly aids the other parties in carrying out the MPC computation. While traditional synchronous MPC protocols provide the weaker abort security guarantee for $t < n$ malicious corruptions (dishonest majority), and only tolerate $t < n/2$ corruptions (honest majority) for stronger security guarantees such as *fairness* [15], Karmakar et al. [29] demonstrated *Asterisk* – a practically efficient helper-aided synchronous MPC protocol that achieves fairness while tolerating either $t < n$ malicious corruptions of the regular parties, or semi-honest corruption of the HP (in particular, the HP does not collude with the malicious regular parties). While helper-aided MPC somewhat bridges the efficiency and security gap between honest and dishonest majority MPC protocols, it does not address asynchronous networks at all. In particular, the protocol of [29] cannot be naturally extended to work over asynchronous networks.

Karmakar et al. [29] motivate the practical applicability of helper-aided MPC with the example use-case of privacy-preserving dark pools [10,2] that allow institutional investors to trade large volumes of financial instruments privately. In this case, the Securities and Exchange Commission (SEC) [10] that governs the operational integrity of dark pool operations can be modeled as a semi-honest HP that does not collude with the buyers/sellers. Another motivating application is privacy-preserving inventory matching [36], where buyers and sellers of financial stocks are matched while ensuring that related stocks can be traded without adversely impacting the market price for either client. In this case, the central institution matching buyers and sellers (such as the bank) can act as the non-colluding HP. We note that for such applications, where a (semi-) trusted central entity is responsible for governing the execution of the protocol, assuming a synchronous network is a very idealized assumption. For example, both dark pools and inventory matching are typically executed over the internet, which is more accurately captured by an asynchronous network. Hence, while *Asterisk* enables practically efficient dishonest majority MPC with strong security guarantees, its restriction to synchronous networks falls short of capturing real-world deployment scenarios for its target applications.

In this paper, we ask the following question: *Can we design practically efficient helper-aided MPC protocols with strong security guarantees for asynchronous networks?*

1.1 Our Contributions

Towards bridging the gap between synchronous and asynchronous MPC protocols, we design practically efficient helper-aided MPC protocols that achieve the best-of-both-worlds network model with strong security and termination guarantees of synchronous networks *and* the ability to operate over asynchronous networks. We consider the same helper-aided MPC model as in [27,29] with a designated non-colluding helper party HP and n regular parties, where the adversary is either allowed to passively corrupt the HP, or maliciously corrupt a subset of the regular parties (we expand more on this corruption model in the subsequent overview). We consider both honest and dishonest majority of malicious corruptions of the regular parties.

- In the honest majority setting, we present a helper-aided n -party AMPC protocol called **Pollux** that achieves the strong security guarantee of *fairness* while relying only on the existence of simple and efficiently implementable symmetric-key cryptographic primitives implied by any one-way function. The HP essentially allows us to bridge the gap between the known corruption thresholds for fairness in synchronous networks ($t < n/2$) and asynchronous networks ($t < n/3$).
- In the dishonest majority setting, we present a helper-aided, n -party AMPC protocol called **Castor** that also achieves *fairness* relying on OT. Notably, in this case, the HP allows bridging a much larger gap between the known corruption thresholds (even for the weaker guarantee of abort security) between synchronous networks ($t < n$) and asynchronous networks ($t < n/3$).
- We show that the use of OT in our dishonest-majority construction is, in fact, necessary. Specifically, we prove that any helper-aided AMPC protocol in the dishonest majority setting—even one that only provides the weaker guarantee of abort security—implies the existence of OT.

Both **Castor** and **Pollux** operate in the well-studied offline/preprocessing-online paradigm for MPC protocols with input-independent (but function-dependent) preprocessing. We present prototype implementations of both protocols and demonstrate their practical efficiency via experimental evaluations. We expand further on our results below. A more detailed treatment of related work appears in Section 1.3.

1.2 Overview of Our Results

As already mentioned, our results are in the model of helper-aided MPC from [27,29] with n regular parties and a designated helper party HP, where the adversary is either allowed to passively corrupt the HP, or maliciously corrupt a subset of the regular parties. The key difference between our model and that of [27,29] is that we consider asynchronous networks, while the treatment in [27,29] was entirely restricted to synchronous networks.

On the Adversarial Model. We note here that the authors of [29] proved that an either-or adversarial corruption model is, in fact, necessary for HP-aided MPC to satisfy fairness in the dishonest majority setting. In particular, it is impossible to achieve fairness in a stronger corruption model that allows two non-colluding adversaries to simultaneously corrupt the HP and a dishonest majority of the regular parties, respectively. While [29] proved this necessity for synchronous networks, it also holds true for asynchronous networks. We adopt the same adversarial model in this paper since we target fairness as the desired guarantee for our constructions.

We now describe our results at a high-level below.

HP-Aided Honest Majority AMPC. As our first contribution, we present **Pollux**—a helper-aided, honest majority n -party AMPC protocol that achieves fairness. At a high level, **Pollux** uses the HP to effectively bridge the gap between the known corruption thresholds for fairness in synchronous networks ($t < n/2$) and asynchronous networks ($t < n/3$). A bit more concretely, **Pollux** follows the standard preprocessing-online paradigm, and leverages Shamir secret sharing, with the helper party HP responsible for generating and distributing the preprocessing data. In the online phase, all communication is routed through HP. This design offers two key advantages: first, it provides broadcast functionality without incurring additional overhead; second, it enables the use of the P_{King} -based approach from [20]. Notably, prior asynchronous protocols were unable to adopt this P_{King} strategy due to the inherent challenges of asynchrony. Through our construction of **Pollux**, we show, for the first time, how to overcome this barrier by leveraging the HP. Interestingly, HP can also be leveraged to achieve strong termination guarantees (as parties can afford to wait for messages from the semi-honest HP). See Section 3 for the detailed construction of **Pollux**.

HP-Aided Dishonest Majority AMPC. We then focus on the dishonest majority setting. As our second contribution, we present a helper-aided, dishonest majority n -party AMPC protocol **Castor** that achieves fairness assuming a (synchronous) maliciously secure OT protocol. Notably, **Castor** establishes the power of the HP in bridging a major gap between the known corruption thresholds (even for the weaker guarantee of abort security) for synchronous networks ($t < n$) and asynchronous networks ($t < n/3$). However, the most remarkable feature of **Castor** is that it achieves not just abort security but fairness, which is unachievable in the dishonest majority setting even over synchronous networks [15].

Technically, **Castor** again follows the standard preprocessing-online paradigm, where the input-independent (but function-dependent) preprocessing phase handles the bulk of the communication and computation, generating correlated randomness for a highly efficient online phase. The design of **Castor** combines techniques from both honest-majority and dishonest-majority MPC protocols: it employs replicated secret sharing (from honest-majority settings) and integrates triple sacrifice and information-theoretic MACs (from dishonest-majority constructions). Crucially, unlike traditional dishonest-majority protocols which require $O(n^2)$ OTs per multiplication, **Castor** reduces this to just $O(n)$ OTs. Specifically, we introduce a new *multi-sender single-receiver OLE* (Oblivious Linear Evaluation, which can be built using OTs) approach, in which the HP acts as the sole receiver while each of the other parties acts as a sender. This design allows the protocol to proceed even if only one regular party is available, without compromising privacy and while also maintaining efficiency. This yields not only a protocol that achieves the optimal corruption threshold but also has the strong guarantee of fairness and termination, which is highly desirable in many practical applications. The detailed construction of **Castor** appears in Section 4.

Necessity of OT. We demonstrate that the use of OT in the above construction is, in fact, necessary. In Section 4.4, we establish that any helper-aided AMPC (with abort security) that tolerates malicious corruption of a majority of the regular parties implies the existence of a synchronous oblivious transfer (OT) protocol. At a high level, our approach is as follows: we construct a two-party OT protocol given an n -party HP-aided dishonest majority AMPC protocol that securely computes a specific function: the helper HP provides inputs (m_0, m_1) , party P_1 provides a bit b , and only P_1 receives the output m_b . In our reduction, the sender emulates the role of HP, and the receiver emulates P_1 . We further leverage the asynchronous nature of the original protocol – specifically, the behavior of the network adversary and the remaining parties – to transform the asynchronous execution into a synchronous OT protocol.

We remark that both **Castor** and **Pollux** allow for input deprivation, which is an inherent problem in asynchronous protocols, meaning that the computation may ignore up to t honest parties’ inputs (we argue that it is inherent also for helper-aided asynchronous MPC in Appendix C). However, our constructions have the desired property that all honest parties’ inputs whose messages were delivered in the first round would be included in the computation. Therefore, in practice, one could imagine the protocol specifying a generous waiting time just for the first step, and subsequently, the parties only wait for $n - t$ messages in each step.

Implementation and Experimentation. We present prototype implementations of both **Pollux** and **Castor**, and benchmark their performance when evaluating circuits with 10^6 multiplication gates, for varying depth ($d \in \{10, 100\}$) and varying number of parties ($n \in \{5, 10, 20, 30\}$). Our experimental results are presented in Section 5. We illustrate that both the offline and online phases of **Pollux** are highly efficient, and scale smoothly for computations involving circuits with large depth, as well as for large numbers of parties. For **Castor**, we illustrate that most of the heavy computation can be offloaded to the preprocessing phase, which allows the online phase to be efficient and scalable.

We compare the running times of the online phases for **Pollux** and **Castor** with that of an implementation of **Asterisk** [29] over asynchronous networks, benchmarked across a varying number of delayed parties. We note that while **Asterisk** uses purely symmetric-key cryptographic primitives and was demonstrated to be highly efficient for synchronous networks in [29], **Pollux** and **Castor** outperform it over asynchronous networks. Even in the absence of network delays, the online phases of **Pollux** and **Castor** are $3\times$ and $5\times$ faster than that of **Asterisk**. In addition, in the presence of network delays, the performance degradation of **Asterisk** is significantly larger than that of **Pollux** and **Castor**. This illustrates the importance of designing dedicated protocols with built-in mechanisms for handling network delays inherent to the asynchronous setting.

1.3 Related Work

The work closest to our model is that of [23], which considers a star-like network topology where multiple relay nodes facilitate communication among the parties executing an MPC protocol. However, their model assumes a synchronous network where a network adversary can delay messages from some parties by a bounded number of rounds. This leads to two primary inefficiencies: (1) the protocol incurs round-based

delays inherent to synchronous settings, and (2) to maintain optimal resilience, all parties must wait for the slowest messages to arrive, exacerbating latency.

In contrast, a wide range of asynchronous MPC (AMPC) protocols exist in the literature. Several works such as [6,34,19,33,14,24,28,1] provide information-theoretic constructions. Of these, [1] achieves linear communication complexity of $O(n)$ elements per multiplication gate, albeit with an additive overhead of $O(n^5)$ elements.

Other works—including [25,26,12,38,13,16,7]—improve communication efficiency by employing various cryptographic primitives, such as additive homomorphic encryption, homomorphic commitments, somewhat homomorphic encryption, and fully homomorphic encryption. Notably, [12] achieves linear communication using additive-homomorphic encryption under the atomic-send model for $t < (1 - \epsilon)n/3$. The recent work of [32] gives an AMPC protocol with worst-case communication of $O(n^2)$ elements per multiplication, and $O(n)$ elements in the optimistic case, relying solely on lightweight cryptography. Similarly, [3] presents a lightweight AMPC protocol achieving linear communication per multiplication gate. However, all these protocols require the honest majority threshold $t < n/3$.

As is well-known (and discussed in this work), even computationally secure MPC protocols with only abort security must satisfy the bound $n \geq 3t + 1$. The work of [4] circumvents this limitation by assuming an input-distribution oracle (realized via a single synchronous broadcast round) and employing threshold homomorphic encryption. Building on the ideas of [18], their construction achieves MPC with $n \geq 2t + 1$. However, this comes with notable trade-offs: the protocol must evaluate the circuit n times, once for each party acting as the distinguished party (analogous to the P_{King} in [20]), and it incurs significant computational overhead due to the use of threshold encryption.

In this work, we consider a recently introduced model [27,29] of HP-aided MPC. [29] demonstrated that it can not only be used to circumvent the known impossibility but also improve the efficiency significantly. However, their work assumes that the parties are connected over a synchronous network.

2 Background and Preliminaries

Notations and Model. We use $\mathcal{P} = \{P_1, \dots, P_n\}$ to denote the set of computational parties, of which up to t parties may be corrupted by a malicious adversary $\mathcal{A}$. In addition, we assume a semi-honest helper party HP, which participates in the protocol and may or may not have inputs or outputs. The helper HP does not collude with the adversary $\mathcal{A}$. Further, we consider that every pair of parties in $\mathcal{P} \cup \{\text{HP}\}$ is connected by a secure and authenticated communication channel. The underlying network is asynchronous; that is, messages may be delayed by an arbitrary but unknown finite time. Moreover, messages may arrive in an arbitrary order. The only guarantee is that all the messages sent by an honest party will eventually be delivered.

We prove the security of our protocols based on the standard real world / ideal world paradigm [8]. Essentially, the security of a protocol is analyzed by comparing what an adversary can do in the real execution of the protocol to what it can do in an ideal execution, that is considered secure by definition (in the presence of an incorruptible trusted party). For a more detailed description of the security model, see Appendix B.1.

Network model. As discussed, we consider that parties and HP are connected via pairwise private and authenticated communication channels, and the network is asynchronous. To model this asynchrony, we use the formalization from [9], which we elaborate in Appendix B.2.

Shared key setup. Our protocols require a shared key setup. We denote by k_{all} the PRF key shared among all parties in $\mathcal{P}$ and HP, by k_i the PRF key shared between P_i and HP, and by $k_{\mathcal{P}}$ the PRF key shared among parties in $\mathcal{P}$ but hidden from HP. Details are deferred to Appendix B.3.

Oblivious Linear Evaluation (OLE). Our dishonest majority construction, *Castor*, relies on the Oblivious Linear Evaluation (OLE) functionality. This is a 2-party functionality where the sender S holds inputs a and c , and the receiver R holds input b . S receives no output, while R obtains $d = a \cdot b - c$. As shown in SPDZ2k [17], this functionality can be efficiently realized using OT.

3 Pollux: HP-aided Honest Majority AMPC

In this section, we present Pollux, a novel semi-honest helper-aided AMPC protocol in the honest majority setting. Pollux overcomes the impossibility of achieving AMPC with more than one-third corrupt parties (Section A) by leveraging a semi-honest helper. It supports n parties with $n \geq 2t + 1$, tolerates up to t corruptions, and guarantees fairness without relying on expensive cryptographic primitives. Built using Shamir secret sharing and information-theoretic MACs, Pollux follows a preprocessing-online paradigm, achieving a communication cost of $4t + 4$ elements in the preprocessing phase and $2n$ elements and 2 rounds per multiplication gate in the online phase.

3.1 Sharing Semantics

Pollux is designed in the preprocessing-online paradigm similar to [29,11,35] where a secret sharing (denoted by $\llbracket \cdot \rrbracket$) involves two primary components, a mask and the masked secret. To generate this, we use Shamir secret sharing to share the (input-independent) masks in the preprocessing phase. We further obtain authenticated Shamir sharing of these, denoted by $\langle \cdot \rangle$, by obtaining a MAC tag on these shared masks. This is required to ensure security against a malicious adversary corrupting up to t parties in $\mathcal{P}$. The (input-dependent) masked value is generated in the online phase to complete the $\llbracket \cdot \rrbracket$ -sharing. Below we present the notations for secret sharing.

- $[v]$: We use $[v]$ to represent a secret v that is Shamir shared among the parties in $\mathcal{P}$, that is each P_i holds $[v]_i$ such that there exists a t degree polynomial $f_v(x)$ such that $f_v(0) = v$ and $f_v(i) = [v]_i$, for all $i \in [n]$. Further, in our case, HP holds $f_v(i)$ for all $i \in [1, n]$.
- $\langle v \rangle$: We consider authenticated Shamir secret sharing represented by $\langle \cdot \rangle$ that is used in the preprocessing phase and is defined as follows. Let $t_v = \Delta \cdot v$ be the tag on v , where Δ is the global MAC key. Authenticated Shamir sharing comprises of the Shamir sharing of the value v and its tag t_v . Let $\langle v \rangle_i$ be the share held by P_i then $\langle v \rangle_i = ([v]_i, [t_v]_i)$ for all $i \in [1, n]$.
- $\llbracket v \rrbracket$: We use $\llbracket v \rrbracket$ in the online phase to share a secret v , where the $\langle \delta_v \rangle$ of a random mask δ_v is generated in the preprocessing phase and the masked value $m_v = v + \delta_v$ is computed in the online phase. All parties excluding HP hold m_v in clear. Let $\llbracket v \rrbracket_i$ be the share of P_i , then $\llbracket v \rrbracket_i = (m_v, \langle \delta_v \rangle_i)$ for all $i \in [1, n]$.

3.2 The Preprocessing Phase

We assume that for all $P_i \in \mathcal{P}$, HP and P_i have a common key k_i . In this phase, HP prepares the preprocessing data, which is primarily the secret sharing of masks corresponding to each wire in the circuit. In contrast to [29], HP requires to communicate appropriate preprocessing data to more than one party. In [29], the mask is additively shared, allowing parties and HP to pick $n - 1$ shares out of the n shares uniformly at random. Whereas, Pollux facilitates computation over an asynchronous network where it is impossible to distinguish between honest parties that are slow and corrupt parties that do not communicate. Due to this, messages from up to t parties could be delayed indefinitely. Thus, it requires us to work with a secret sharing scheme, which allows the protocol to make progress even if t messages are not delivered. Therefore, Pollux works with Shamir secret sharing with the threshold t . To share a secret using Shamir secret sharing at most t values can be uniformly chosen. Thus, HP prepares the preprocessing data for the circuit and generates the shares for t parties non-interactively. Then it computes the shares of the remaining $t + 1$ parties and communicates it to them. Since HP is not malicious, it certainly communicates the messages and therefore it is guaranteed that all the parties eventually receive their respective shares. Furthermore, since no other (possibly malicious) party communicates to generate the preprocessing data, it does not need any validity check. Below we provide an overview of how preprocessing data is generated and shared.

Generating Shamir sharing of a random value v . To generate $[v]$ of a random value v , parties use the shared key setup as shown in Fig. 1.

Generating the authenticated Shamir sharing of a random value v . To ensure security against malicious adversary, in addition to Shamir sharing of the value v , HP computes tag $t_v = \Delta \cdot v$ on it. Further,

parties obtain Shamir sharing of the tag to complete the authenticated sharing. That is, parties obtain $\langle v \rangle$ by computing $[v]$ and $[t_v]$.

Protocol Π_{RandSSS}

Input: $\forall i \in [1, n]$, P_i holds a PRF key k_i . HP holds $\{k_i\}_{i \in [1, n]}$.

Output: Parties output $[v]$ where each P_i holds $[v]_i$.

Protocol:

- $\forall i \in [1, t]$, P_i and HP sample $[v]_i$ using k_i .
- HP generates a degree- t polynomial $f_v(x)$ such that $f_v(0) = v$ and $f_v(i) = [v]_i$, $\forall i \in [1, t]$.
- $\forall i \in [t+1, n]$, HP sends $f_v(i)$ to P_i .
- $\forall i \in [1, n]$, P_i outputs $[v]_i$.

Fig. 1: Generating Shamir shares of a random value

Protocol Π_{RandASSS}

Input: $\forall i \in [1, n]$, P_i holds a PRF key k_i . HP holds $\{k_i\}_{i \in [1, n]}$. Parties also hold $[\Delta]$ of the global MAC key Δ .

Output: Parties output $\langle v \rangle$ where each P_i holds $\langle v \rangle_i$.

Protocol:

- HP computes the tag $t_v = \Delta \cdot v$.
- Parties invoke Π_{RandSSS} (Fig. 1) to share $[v]$ and $[t_v]$ such that each P_i holds $([v]_i, [t_v]_i)$ and HP holds $\{[v]_i, [t_v]_i\}_{i \in [1, n]}$.
- $\forall i \in [1, n]$, P_i outputs $\langle v \rangle_i = ([v]_i, [t_v]_i)$.

Fig. 2: Generating authenticated Shamir shares of a random value

Input gates. Let P_d be an input provider and v be the corresponding input. Then HP and P_d sample a random mask δ_v using their common key k_d . Then parties and HP perform Π_{RandASSS} to obtain $\langle \delta_v \rangle$.

Addition gates. Due to the linearity of $\langle \cdot \rangle$ -sharing, parties can locally compute the authenticated sharing $\langle \delta_z \rangle$ of the output mask δ_z as the sum of the input masks, i.e., from $\langle \delta_x \rangle$ and $\langle \delta_y \rangle$, where δ_x , δ_y , and δ_z are the masks corresponding to the input and output wires of the addition gate, respectively.

Multiplication gates. Let x, y be the inputs of a multiplication gate. Then the corresponding masks δ_x and δ_y are $\langle \cdot \rangle$ -shared. Observe that HP has δ_x and δ_y in clear. It computes $\delta_{xy} = \delta_x \cdot \delta_y$ and samples δ_z uniformly at random. Parties and HP then execute Π_{RandASSS} to obtain $\langle \delta_{xy} \rangle$ and $\langle \delta_z \rangle$.

Output gates. To ensure fairness, for all the output gates, $\langle \cdot \rangle$ -sharing of the masks are generated in such a way that HP gets the random mask in clear. Going forward, HP will send this mask to all the parties to reconstruct the outputs.

Protocol $\Pi_{\text{Pollux}}^{\text{Preproc}}$

Input: $\forall i \in [1, n]$, P_i and HP have the circuit C , common PRF keys $\{k_i, k_P, k_{\text{all}}\}$ generated in the setup phase.

Output: $\forall i \in [1, n]$, P_i outputs preproc_i and HP outputs $\text{preproc}_{\text{HP}}$.

Protocol:

- **MAC key:** HP samples a MAC key Δ and together with parties in $\mathcal{P}$, it invokes Π_{RandSSS} (Fig. 1) to generate $[\Delta]$ such that each P_i holds the share $[\Delta]_i$ of the global MAC key Δ .
- **Input gates:** For each input gate v with an input provider P_d , HP and P_d sample a random mask δ_v . Then HP and parties in $\mathcal{P}$ invoke Π_{RandASSS} (Fig. 2) to generate $\langle \delta_v \rangle$ such that δ_v is known to P_d and each P_i outputs $\langle \delta_v \rangle_i$.
- **Addition gates:** For each addition gate with inputs x and y , parties and HP locally compute $\langle \delta_{x+y} \rangle = ([\delta_x] + [\delta_y], [t_x] + [t_y])$.
- **Multiplication gates:** For each multiplication gate $z = xy$, HP computes $\delta_{xy} = \delta_x \cdot \delta_y$ and samples a random δ_z . Then parties in $\mathcal{P}$ and HP invoke Π_{RandASSS} (Fig. 2) to generate $\langle \delta_{xy} \rangle, \langle \delta_z \rangle$.

- *Output gates:* For each output gate z , parties in $\mathcal{P}$ and HP invoke Π_{RandASSS} (Fig. 2) to generate $\langle \delta_z \rangle$.

Fig. 3: The preprocessing phase of Pollux

3.3 The Online Phase

In the online phase, parties in $\mathcal{P}$ reconstruct the masked value corresponding to the secrets of each wire to complete the $\llbracket \cdot \rrbracket$ -sharing. To have privacy from a semi-honest HP who has the complete preprocessing data, it is important that the masked value corresponding to a wire is not known to HP.

Input gates. Let P_d be the party with input v . The random mask, δ_v , for v is generated in the preprocessing phase such that parties in $\mathcal{P}$ hold the authenticated Shamir sharing of $\langle \delta_v \rangle$. Moreover, HP and P_d hold all the shares as well as δ_v in clear. All parties in $\mathcal{P}$ hold a common key $k_{\mathcal{P}}$, using which they sample a random value r_v . P_d computes the masked value $m_v = v + \delta_v$. It sends the padded masked value $q_v = m_v + r_v$ to HP, who then forwards q_v to all parties. Upon receiving q_v from HP, all parties further obtain the masked value by subtracting r_v from q_v . Then each party $P_i \in \mathcal{P}$ outputs $\llbracket v \rrbracket_i = (m_v, \langle \delta_v \rangle_i)$.

Protocol $\Pi_{\llbracket \cdot \rrbracket\text{-Sh}}^{\text{Pollux}}$

Input:

- Every input provider/dealer P_d , has input v and mask δ_v .
- $\forall i \in [1, n]$, P_i has the input gate g_{in} , $\langle \delta_v \rangle_i$ generated in Fig. 3 for g_{in} and the common PRF key $k_{\mathcal{P}}$.

Output: $\forall i \in [1, n]$, P_i outputs $\llbracket v \rrbracket_i = (m_v, \langle \delta_v \rangle_i)$, where $m_v = v + \delta_v$.

Protocol:

- All $P_i \in \mathcal{P}$ sample a random value r_v using $k_{\mathcal{P}}$.
- P_d computes $q_v = v + \delta_v + r_v$ and sends it to HP.
- Upon receiving q_v values corresponding to (at least) $n - t$ parties' inputs, HP forwards these to all parties.
- $\forall i \in [1, n]$, P_i obtains $m_v = q_v - r_v$.
- $\forall i \in [1, n]$, P_i outputs $\llbracket v \rrbracket_i = (m_v, \langle \delta_v \rangle_i)$.

Fig. 4: Online phase of input sharing of Pollux

Addition gates. Due to the homomorphic property of the secret sharing scheme, each party locally obtains the sharing of the output wire of the addition gates.

Multiplication gates. Consider a multiplication gate with inputs x, y and output z such that parties hold $\llbracket x \rrbracket, \llbracket y \rrbracket$. Moreover, from the preprocessing phase, parties already hold $\langle \delta_{xy} \rangle, \langle \delta_z \rangle$ where $\delta_{xy} = \delta_x \cdot \delta_y$ such that δ_{xy}, δ_z are known to HP. In the online phase, $P_i \in \mathcal{P}$ computes $\langle m_z \rangle_i = m_x \cdot m_y - m_x \cdot \langle \delta_y \rangle_i - m_y \cdot \langle \delta_x \rangle_i + \langle \delta_{xy} \rangle_i + \langle \delta_z \rangle_i$.

Recall that $\langle m_z \rangle_i$ comprises of two components: $[m_z]_i$ and $[t_{m_z}]_i$. All parties in $\mathcal{P}$ sample a common random value r_z using the common key $k_{\mathcal{P}}$. Each P_i sends $[m_z]_i + r_z$ to HP. Upon receiving (at least) $n - t$ shares from the parties, HP reconstructs $m_z + r_z$ and sends it to all parties. For each $i \in [1, n]$, upon receiving $m_z + r_z$ from HP, P_i obtains m_z by subtracting r_z locally. Finally, parties output $\llbracket z \rrbracket$.

Protocol $\Pi_{\text{mult}}^{\text{Pollux}}$

Input: $\forall i \in [1, n]$, P_i has the multiplication gate g_{mul} , $\llbracket x \rrbracket_i, \llbracket y \rrbracket_i$ and $\langle \delta_{xy} \rangle_i, \langle \delta_z \rangle_i$ generated in Fig. 3 for g_{mul} , the common PRF key $k_{\mathcal{P}}$ generated in the setup phase and the share of the global MAC key $[\Delta]_i$.

Output: $\forall i \in [1, n]$, P_i outputs $m_z = z + \delta_z$, where $z = x \cdot y$.

Protocol:

- All $P_i \in \mathcal{P}$ sample a random value r_z using $k_{\mathcal{P}}$.
- $\forall i \in [1, n]$, P_i computes
- $[t_{m_z}]_i = [\Delta]_i \cdot (m_x \cdot m_y) - m_x \cdot [t_y]_i - m_y \cdot [t_x]_i + [t_{xy}]_i + [t_z]_i$,

- $[m_z]_i = m_x \cdot m_y - m_x \cdot [\delta_y]_i - m_y \cdot [\delta_x]_i + [\delta_{xy}]_i + [\delta_z]_i$,
- $[q_z]_i = [m_z]_i + r_z$.
- $\forall i \in [1, n]$, P_i sends $[q_z]_i$ to HP.
- HP waits for the messages from (at least) $n - t$ parties. It then reconstructs q_z and sends it to all parties.
- $\forall i \in [1, n]$, P_i obtains $m_z = q_z - r_z$.
- $\forall i \in [1, n]$, P_i outputs $(\llbracket z \rrbracket_i, [t_{m_z}]_i)$, where $\llbracket z \rrbracket_i = (m_z, \langle \delta_z \rangle_i)$.

Fig. 5: Online phase of multiplication of Pollux

Verification. Note that the gate-by-gate evaluation is done optimistically; however, a corrupt party can potentially introduce errors in the reconstructed terms. Hence, we need to verify the correctness of each reconstructed m_z . Specifically, it suffices to check whether $[t_{m_z}] - m_z \cdot [\Delta]$ is a sharing of 0. For efficiency, we combine the verification checks for many multiplication gates into a single aggregated check, which is performed at the end of the computation, just before output reconstruction. This approach amortizes the cost of verification. To combine the checks for ℓ multiplication gates into one, the parties proceed as follows:

- All parties (including HP) sample a random vector $\rho = (\rho_0, \rho_1, \dots, \rho_\ell) \leftarrow \mathbb{F}^{\ell+1}$ using a common coin functionality $\mathcal{F}_{\text{Rand}}$ that enables them to sample common random values such that an adversarial party can guess the sampled value with only a negligible probability. To realize $\mathcal{F}_{\text{Rand}}$, HP samples a key k_{ver} and sends it to all parties. Each party waits to receive k_{ver} , and then evaluates a PRF with key k_{ver} on a common input ctr . Note that we cannot use k_{all} for $\mathcal{F}_{\text{Rand}}$, since an adversarial party would already know the key and could compute ρ . Hence, the soundness of the verification check would no longer hold. Since k_{ver} is unknown to the parties, a malicious party can only guess ρ with negligible probability, and hence can only cheat and pass the verification with negligible probability.
- All parties (excluding HP) sample a random degree- t sharing of 0, i.e., each party $P_i \in \mathcal{P}$ computes its share α_i such that there exists a degree- t polynomial $f(x)$ with $f(i) = \alpha_i$ and $f(0) = 0$.

Let $\{m_{z_j}\}_{j \in [1, \ell]}$ be the reconstructed masked values known to all parties, and let $\{[t_{m_{z_j}}]_i\}_{j \in [1, \ell]}$ be the corresponding shares of the tags held by P_i . For each $j \in [1, \ell]$, P_i computes: $[\omega_{z_j}]_i = [t_{m_{z_j}}]_i - m_{z_j} \cdot [\Delta]_i$ and then sends $[\omega_{z_j}]_i$ to HP where $[\omega_z]_i = \rho_0 \cdot \alpha_i + \sum_{j \in [\ell]} \rho_j \cdot [\omega_{z_j}]_i$.

Upon receiving $n - t$ such values, HP reconstructs ω_z and checks whether $\omega_z = 0$. The result of the verification is then sent to all parties.

Output reconstruction. If the output of the MAC verification is 1, then HP sends the masks of the output wires to all parties. Let z denote an output that is $\llbracket \cdot \rrbracket$ -shared. Each party in $\mathcal{P}$ waits to receive δ_z from HP and then locally computes the final output as $z = m_z - \delta_z$. Since HP is honest, all the parties either receive the mask (if the verification passes) or receive $\perp$ (if the verification fails). In both the cases all the parties terminate, thus ensures strong termination guarantee.

Protocol $\Pi_{\text{verify}}^{\text{Pollux}}$

Input: $\forall i \in [1, n]$, P_i has the reconstructed masked values $\{m_{z_1}, m_{z_2}, \dots, m_{z_\ell}\}$, and $[t_{m_{z_j}}]_i$ as the share of MAC on m_{z_j} , for $j \in [1, \ell]$. P_i also has the common PRF key $k_{\mathcal{P}}$ generated in the setup phase and the share of the global MAC key $[\Delta]_i$.

Output: All parties and HP output 1 if the verification passes, else 0.

Protocol:

- HP samples a new PRF key k_{ver} and sends it to all the parties in $\mathcal{P}$.
- $\forall i \in [1, n]$, P_i does the following.
 - Computes $[\omega_{z_j}]_i = [t_{m_{z_j}}]_i - m_{z_j} \cdot [\Delta]_i$, $\forall j \in [1, \ell]$.
 - Samples $\rho = (\rho_0, \rho_1, \dots, \rho_\ell) \leftarrow \mathbb{F}^{\ell+1}$ using k_{ver} .

- Samples a random sharing of 0 using k_P , i.e., it gets a share α_i such that there exists a degree- t polynomial $f(x)$ such that $f(0) = 0$ and $f(i) = \alpha_i$.
- Sends $[\omega_z]_i = \rho_0 \cdot \alpha_i + \sum_{j=1}^{\ell} \rho_j \cdot [\omega_{z_j}]_i$ to HP.
- Upon receiving messages from (at least) $n - t$ parties, HP reconstructs ω_z and checks if $\omega_z = 0$. If the check fails, then HP sends 0 to all, else it sends 1.

Fig. 6: Amortized MAC-verification of Pollux

Protocol $\Pi_{\text{Online}}^{\text{Pollux}}$

Input:

- For every input provider/dealer P_d , it has input v .
- $\forall i \in [1, n]$, P_i and HP have the circuit C , the preprocessing data generated in Fig. 3 and common PRF keys $\{k_i, k_P, k_{\text{all}}\}$ generated in the setup phase.

Output: $\forall i \in [1, n]$, P_i s outputs $C(x_1, x_2, \dots, x_n)$ or $\perp$.

Protocol:

- *Input gates:* For every input gate of C , parties execute $\Pi_{\llbracket \cdot \rrbracket\text{-Sh}}^{\text{Pollux}}$ (Fig. 4).
- *Addition gates:* For every addition gate of C with inputs x, y and output z , parties and HP locally set $m_z = m_x + m_y$.
- *Multiplication gates:* For every multiplication gate of C , parties execute $\Pi_{\text{mult}}^{\text{Pollux}}$ (Fig. 5).
- *Amortized verification:* At the end of all the multiplication gates, all parties and HP execute $\Pi_{\text{verify}}^{\text{Pollux}}$ (Fig. 6). If parties output 1, then **continue**, else output $\perp$.
- *Output gates:* If the output of *Verification* is 1, then HP sends the mask δ_z for all output gates to all the parties. Else, it sends $\perp$ to all the parties.

Fig. 7: The online phase of Pollux

Security. We defer a detailed discussion on the security of Pollux to Appendix F.1.

4 Castor: HP-aided Dishonest Majority AMPC

In this section, we go beyond honest majority and leverage the helper party to design a *fair* AMPC protocol that tolerates a *dishonest majority*. **Castor** follows a preprocessing-online paradigm. The preprocessing phase involves a communication of $4n + 6$ elements and communication required for evaluating $12n$ OLEs. The online phase involves $2n$ elements and single round per multiplication gate.

It is also in the preprocessing-online paradigm where a function is represented as an arithmetic circuit. Then it is evaluated gate-by-gate in a topological order. The function dependent (input-independent) preprocessing phase generates the required correlated randomness that gets utilized in the subsequent online phase. For reasons that will be clear later, the sharing semantics required in this protocol differ from Pollux and [29] beyond the change in the threshold of sharing. Particularly, in addition to the computational parties, the helper also receives the masked values in the online phase. To allow this while ensuring privacy, the helper cannot possess the complete preprocessing data. Hence, we consider a new sharing semantics to facilitate the computation where the helper and one of the computational parties can complete the computation and any $n - 1$ colluding parties cannot reconstruct a secret shared value. We now discuss the sharing semantics elaborately.

4.1 Sharing Semantics

Analogous to Pollux and [29], we employ masked secret sharing, denoted as $\llbracket \cdot \rrbracket$. In this approach, a random mask is generated during the preprocessing phase, while the masked value that incorporates the secret

is computed in the online phase. The masked value is then made available to all parties, including HP. However, to preserve privacy, the random mask generated in preprocessing phase must not be known to HP. To achieve this, the mask is secret-shared in a way that HP lacks one share, ensuring its privacy from HP. This consequently ensures the privacy of the secret even when HP holds the masked value. To allow parties to complete the protocol, all computational parties hold the share of the mask that HP does not hold. Additionally, each computational party holds a unique share of the mask that is also held by HP. To protect against malicious adversaries, the mask must be authenticated. For this, we use information-theoretic MACs, as in previous constructions. The global MAC key and the corresponding tag of the mask are also secret-shared in a similar manner.

Below we present the notations for secret sharing.

- $[v]$: We define replicated secret sharing of a value v denoted by $[v]$ as follows.⁶ Let $v = \sum_{i=0}^n [v]_i$ where a party P_i 's share is the tuple $([v]_0, [v]_i)$, and HP's share is $([v]_1, \dots, [v]_n)$.
- $\langle v \rangle$: We consider the authenticated replicated secret sharing represented by $\langle \cdot \rangle$ that is used in the preprocessing phase and is defined as follows. Let $t_v = \Delta \cdot v$ be the tag on v , where Δ is the global MAC key. Authenticated replicated sharing comprises of replicated sharing of the value v and its tag t_v . Let $\langle v \rangle_0$ be the share of HP and $\langle v \rangle_i$ be the share of P_i , for all $i \in [1, n]$. Then $\langle v \rangle_0 = ([v]_1, \dots, [v]_n, [t_v]_1, \dots, [t_v]_n)$, and $\langle v \rangle_i = ([v]_0, [v]_i, [t_v]_0, [t_v]_i)$ for all $i \in [1, n]$.
- $\llbracket v \rrbracket$: We use $\llbracket v \rrbracket$ in the online phase to share a secret v , where the $\langle \delta_v \rangle$ of a random mask δ_v is generated in the preprocessing phase and the masked value $m_v = v + \delta_v$ is computed in the online phase. All parties including HP hold m_v in clear. Let $\llbracket v \rrbracket_0$ be the share of HP and $\llbracket v \rrbracket_i$ be the share of P_i , for all $i \in [1, n]$. Then $\llbracket v \rrbracket_0 = (m_v, \langle \delta_v \rangle_0)$, and $\llbracket v \rrbracket_i = (m_v, \langle \delta_v \rangle_i)$ for all $i \in [1, n]$.

4.2 The Preprocessing Phase

In the preprocessing phase, parties generate the required correlated randomness which will be utilized in the online phase. Specifically, parties generate the replicated secret sharing of a global MAC key $[\Delta]$ that will be used for authentication. Further, parties generate the authenticated replicated sharing of the mask $\langle \delta_v \rangle$ corresponding to each wire v in the circuit with respect to the key Δ . Below we will discuss how to generate $[\cdot]$ and $\langle \cdot \rangle$ of a secret v .

Generating the replicated sharing of a random value v . To generate the $[v]$ of a value random v , parties use the shared key setup as shown in Fig. 8.

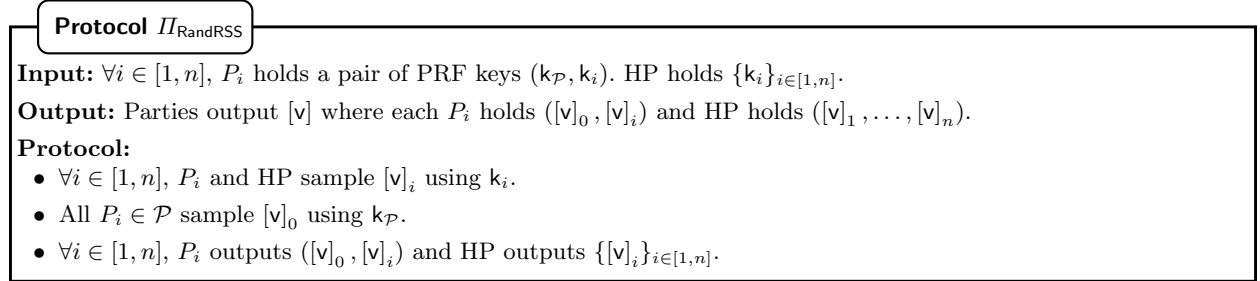


Fig. 8: Generating replicated shares of a random value

This protocol is used to generate the replicated sharing of a global MAC key Δ , and also utilized in generating authenticated replicated shares of the masks for each wire as discussed next. Moreover, by using an appropriate key setup, parties can also create $[v]$ of a value v that is known to a specific party. Specifically,

⁶ This sharing semantics differs from traditional replicated secret sharing, which assumes a symmetric structure among parties. In our construction, the roles are inherently asymmetric—particularly, the role of HP differs significantly from that of the other parties. Nonetheless, since the shares are still replicated among the parties, we refer to this scheme as replicated secret sharing.

if P_d requires to know the value v , then parties use a key setup such that the keys are known to P_d and it samples each share along with the other parties as described in Fig. 8.

Generating the authenticated replicated sharing of a random value v . For generating $\langle v \rangle$ of a random value v , parties first obtain a replicated secret sharing of $[v]$ of a random value v using the procedure above. Note that a global key Δ is also replicated shared. Parties compute $[t_v] \leftarrow \text{mult}([\Delta], [v])$ and output $\langle v \rangle = ([v], [t_v])$. Parties can also compute $\langle v \rangle$ of a value v known to a specific party by using the same procedure with an appropriate shared key setup as described for the prior case.

Protocol Π_{RandRSS}
Input: $\forall i \in [1, n]$ holds a pair of PRF keys (k_P, k_i). HP holds $\{k_i\}_{i \in [1, n]}$. Parties also hold $[\Delta]$ of the global MAC key Δ. Output: Parties output $\langle v \rangle$ where each P_i holds $\langle v \rangle_i$. Protocol: • Parties invoke Π_{RandRSS} (Fig. 8) to obtain $[v]$ such that each P_i holds $([v]_0, [v]_i)$ and HP holds $\{[v]_i\}_{i \in [1, n]}$. • Parties invoke $\Pi_{\text{mult-RSS}}$ (Fig. 10) with inputs $[\Delta]$ and $[v]$ to obtain $[t_v]$ where $t_v = \Delta \cdot v$. Each P_i holds $([t_v]_0, [t_v]_i)$ and HP holds $\{[t_v]_i\}_{i \in [1, n]}$. • $\forall i \in [1, n]$, P_i outputs $\langle v \rangle_i = ([v]_0, [v]_i, [t_v]_0, [t_v]_i)$ and HP outputs $\langle v \rangle_0 = \{([v]_i, [t_v]_i)\}_{i \in [1, n]}$.

Fig. 9: Generating authenticated replicated shares of a random value

Input gates. Let P_d be an input provider and v be the corresponding input. Then parties in $\mathcal{P}$ and HP prepare $\langle \delta_v \rangle$ of a value δ_v known to P_d . For this, parties generate a $[\cdot]$ -sharing of δ_v such that P_d knows δ_v in clear. This can be done non-interactively using common key setup as described. Parties further authenticate $[\delta_v]$, and output $\langle \delta_v \rangle$.

Addition gates. Note that, due to the linearity of $\langle \cdot \rangle$ -sharing, the parties can locally compute the authenticated sharing $\langle \delta_z \rangle$ of the output mask δ_z as the sum of the input masks.

Multiplication gates. To ensure a fast online phase, we use the approach of [35]. Given $\langle \cdot \rangle$ -sharing of a random multiplication triple $(\delta_x, \delta_y, \delta_{xy})$ and a random mask δ_z , the online phase requires to compute the masked value m_z corresponding to the output z . Thus, in the preprocessing phase, parties require to generate $\langle \delta_{xy} \rangle$ and $\langle \delta_z \rangle$ given $\langle \delta_x \rangle$ and $\langle \delta_y \rangle$ and δ_z is uniformly sampled.

Output gates. For all output gates, $\langle \cdot \rangle$ -sharing of the masks are generated. In the online phase, the mask is opened, and combined with the masked value to reveal the output.

Triple sacrifice. Unlike [29], where HP alone generated the preprocessing data, our protocol involves both HP and the computational parties. As a result, the adversary may introduce an additive error ϵ during triple generation, yielding authenticated shares of $(\delta_x, \delta_y, \delta_{xy} + \epsilon)$. To ensure correctness, we adopt the triple sacrifice technique from prior works [21, 31, 30, 5]. Specifically, we generate correlated triple pairs $(\langle x \rangle, \langle y \rangle, \langle xy \rangle)$ and $(\langle x \rangle, \langle \hat{y} \rangle, \langle \hat{x}\hat{y} \rangle)$ and perform the following check: a random r is chosen, and $\rho = r \cdot y - \hat{y}$ is partially opened via HP, followed by computing and revealing $\tau = r \cdot xy - \hat{x}\hat{y} - \rho \cdot x$. If $\tau \neq 0$, parties abort. Since these openings are unauthenticated, the adversary may cheat. To catch such behavior efficiently, we apply a MAC check on a random linear combination of all ρ s and τ s across the circuit. This ensures soundness with overwhelming probability while avoiding high communication cost. Importantly, ρ does not leak information as $(\hat{y}, \hat{x}\hat{y})$ are sacrificed and unused thereafter.

Multiplication of two $[\cdot]$ -shared values. Here we will discuss how to multiply two replicated shared secrets, as this protocol will be used to generate multiplication triple and authenticate a replicated shared value. The protocol $\Pi_{\text{mult-RSS}}$ with inputs $[x]$ and $[y]$ requires to compute $(n+1)^2$ terms, $\sum_{i=0}^n \sum_{j=0}^n [x]_i \cdot [y]_j$. Out of these, the terms of the form $[x]_i \cdot [y]_i$ can be computed by at least two parties locally. Then the terms of the form $[x]_i \cdot [y]_j$, for $i \neq j \neq 0$ can be computed and distributed by HP, as HP is not malicious, parties can wait for the shares of these terms from HP. However, the terms of the form $[x]_i [y]_0$ and $[x]_0 [y]_i$ are the most difficult to handle. P_i can compute them, however due to the asynchronous network or malicious

behaviour it may not be properly shared among the parties. Thus, for all such terms HP and P_j , for $j \neq i$ requires to perform OLEs. This requires executing $2n(n-1)$ OLEs for one $\Pi_{\text{mult-RSS}}$. Moreover, this approach is insecure. Consider the following adversarial strategy. Suppose corrupt parties P_1 and P_2 participate in computing $[x]_0 \cdot [y]_3$. They can choose their inputs to the respective OLEs as $[x]_0$ and $[x]_0 + 1$, and receive corresponding outputs u and u' . This would allow them to compute $u' - u = [y]_3$, thereby learning the secret share $[y]_3$ and breaking secrecy.

Multi-sender single-receiver OLE: We introduce a new approach to resolve the above issue that requires lesser number of OLEs per $\Pi_{\text{mult-RSS}}$ execution. All parties and HP run a multi-sender single-receiver OLE where HP acts as the receiver with input $\sum_{i=1}^n [x]_i$ and all the parties emulate the role of the sender with inputs $[y]_0$ and $[z^{(i0)}]_0$, where all the parties excluding HP sample $[z^{(i0)}]_0$ using common key k_P . HP receives u such that $[z^{(i0)}]_0 + u = (\sum_{i=1}^n [x]_i) \cdot [y]_0$. For all $i \in [1, n-1]$, P_i samples $[z^{(i0)}]_i$ with HP using k_i . Finally, HP sends the adjusted share, that is $[z^{(i0)}]_n = u - \sum_{i=1}^{n-1} [z^{(i0)}]_i$ to P_n . Similarly, HP and all the parties compute $(\sum_{i=1}^n [y]_i) \cdot [x]_0$ and obtain the respective shares.

Observe that, here we consider HP as the receiver of the OLE since if we consider the other parties as the receiver then for some of the OLEs termination may not happen due to malicious behavior or delayed network, thus ensuring that those parties receive their respective share gets tricky. Considering HP as receiver addresses the concern that no party misses their share, however it is required to ensure that all OLE instances terminate. To ensure this, all parties wait for HP's messages. On the other hand, HP waits only for one party's message; once it receives the appropriate message from some party, it proceeds to the next round and sends the next round message to all the parties. Finally, if it obtains the output it sends a terminate signal to all the parties. It is crucial to note that once HP receives a message from one party, it discards all the messages of the same round from the other parties. This is slightly different from standard OLEs, thus we introduce a new $n+1$ party functionality $\mathcal{F}_{\text{MSSR-OLE}}$ where n parties with common inputs a and c as sender and the remaining one party with input b acts as the receiver and it receives $a \cdot b - c$. It is easy to realize $\mathcal{F}_{\text{MSSR-OLE}}$ securely using common key setup and standard OLEs. The descriptions of $\mathcal{F}_{\text{MSSR-OLE}}$ and $\Pi_{\text{MSSR-OLE}}$ appear in Fig. 18 and Fig. 19 respectively.

Multiplication of two $\langle \cdot \rangle$ -shared values. We now describe how to multiply two authenticated shared values to generate authenticated multiplication triples $([\delta_x], [\delta_y], [\delta_{xy}])$ for all multiplication gates. Let $\langle x \rangle$ and $\langle y \rangle$ be the inputs to the protocol $\Pi_{\text{mult-ARSS}}$. First, parties execute $\Pi_{\text{mult-RSS}}$ on the replicated shares $[x], [y]$ to obtain $[xy]$, and then again on $[\Delta]$ and $[xy]$ to compute $[t_{xy}]$. The result is the authenticated product $\langle xy \rangle = ([xy], [t_{xy}])$. Additional authenticated values $\langle \hat{\delta}_x \rangle$ and $\langle \hat{\delta}_{xy} \rangle$ are generated similarly for triple sacrifice. Finally, parties perform a triple sacrifice to verify that $([\delta_x], [\delta_y], [\delta_{xy}])$ forms a correct multiplication triple.

Protocol $\Pi_{\text{mult-RSS}}$

Input: $\forall i \in [1, n]$, P_i and HP have $[x], [y]$ and common PRF keys $\{k_i, k_P, k_{\text{all}}\}$ generated in the setup phase.

Output: $\forall i \in [1, n]$, P_i and HP output $[z]$, where $z = x \cdot y$.

Protocol: If $P = \text{HP}$, then it does the following:

- Initializes $[z]_i \leftarrow [x]_i \cdot [y]_i$ for all $i \in [1, n]$.
- It computes $\eta = \sum_{i \neq j} [x]_i \cdot [y]_j$.
- It executes $\mathcal{F}_{\text{MSSR-OLE}}$ as a receiver with input $\sum_{i \in [1, n]} x_i$, where $P_1, \dots, P_n$ are the senders. It receives u .
- It executes $\mathcal{F}_{\text{MSSR-OLE}}$ as a receiver with input $\sum_{i \in [1, n]} y_i$, where $P_1, \dots, P_n$ are the senders. It receives w .
- $\forall i \in [1, n-1]$, it samples random values $[z^{(0i)}]_i$, $[z^{(i0)}]_i$, and $[\eta]_i$ with P_i using k_i .
- It computes $[z^{(i0)}]_n = u - \sum_{i=1}^{n-1} [z^{(i0)}]_i$, $[z^{(0i)}]_n = w - \sum_{i=1}^{n-1} [z^{(0i)}]_i$ and $[\eta]_n = \eta - \sum_{i=1}^{n-1} [\eta]_i$.
- It sends $[z^{(0i)}]_n + [z^{(i0)}]_n + [\eta]_n$ to P_n .
- Updates $[z]_i \leftarrow [z]_i + [z^{(0i)}]_i + [z^{(i0)}]_i + [\eta]_n$, $\forall i \in [1, n]$.

If $P \neq \text{HP}$, it does the following:

- Initializes $[z]_0 \leftarrow [x]_0 \cdot [y]_0$ and $[z]_i \leftarrow [x]_i \cdot [y]_i$.
- All parties excluding HP sample random values $[z^{(i0)}]_0, [z^{(0i)}]_0$ using $k_{\mathcal{P}}$.
- It executes $\mathcal{F}_{\text{MSSR-OLE}}$ as a sender with inputs $y_0, [z^{(i0)}]_0$, where $P_1, \dots, P_n$ are the senders and HP is a receiver.
- It executes $\mathcal{F}_{\text{MSSR-OLE}}$ as a sender with inputs $x_0, [z^{(0i)}]_0$, where $P_1, \dots, P_n$ are the senders and HP is a receiver.
- If $P \neq P_n$, it samples $[z^{(0i)}]_i, [z^{(i0)}]_i$ and $[\eta]_i$ with HP using k_i . Else, receives $[z^{(0i)}]_n + [z^{(i0)}]_n + [\eta]_n$ from HP.
- Updates $[z]_i \leftarrow [z]_i + [z^{(0i)}]_i + [z^{(i0)}]_i + [\eta]_i$ and $[z]_0 \leftarrow [z]_0 + [z^{(0i)}]_0 + [z^{(i0)}]_0$.

Fig. 10: Multiplying replicated shared values

Protocol $\Pi_{\text{mult-ARSS}}$

Input: $\forall i \in [1, n]$, P_i and HP have $\langle x \rangle, \langle y \rangle$ and common PRF keys $\{k_i, k_{\mathcal{P}}, k_{\text{all}}\}$ generated in the setup phase.

Output: $\forall i \in [1, n]$, P_i and HP output $\langle z \rangle$, where $z = x \cdot y$.

Protocol:

Optimistic Multiplication:

- Run $\Pi_{\text{mult-RSS}}$ (Fig. 10) with inputs $[x], [y]$ to obtain $[xy]$.
- Run $\Pi_{\text{mult-RSS}}$ (Fig. 10) with inputs $[xy], [\Delta]$ to obtain $[t_{xy}]$.

Triple Sacrifice:

- Invoke Π_{RandARSS} (Fig. 9) to generate $\langle \hat{y} \rangle$.
- Run $\Pi_{\text{mult-RSS}}$ (Fig. 10) with inputs $[x], [\hat{y}]$ to obtain $[x\hat{y}]$.
- Run $\Pi_{\text{mult-RSS}}$ (Fig. 10) with inputs $[x\hat{y}], [\Delta]$ to obtain $[t_{x\hat{y}}]$.
- Sample a random value r using k_{all} and compute $\langle \rho \rangle = r \cdot \langle y \rangle - \langle \hat{y} \rangle$.
- $\forall i \in [1, n]$, P_i sends $[\rho]_0$ to HP, and HP sends $\tilde{\rho} = \sum_{i=1}^n [\rho]_i$ to all. Then each P_i including HP (after receiving $[\rho]_0$ from a single party) reconstructs $\rho = [\rho]_0 + \tilde{\rho}$.
- Parties compute $\langle \tau \rangle = r \cdot \langle xy \rangle - \langle x\hat{y} \rangle - \rho \cdot x$.
- $\forall i \in [1, n]$, P_i sends $[\tau]_0$ to HP, and HP sends $\tilde{\tau} = \sum_{i=1}^n [\tau]_i$ to all. Then each P_i including HP (after receiving $[\tau]_0$ from a single party) reconstructs $\tau = [\tau]_0 + \tilde{\tau}$.
- If $\tau = 0$, then parties and HP proceed to the verification, otherwise all **abort**.

(One-shot) Verification: Let $\{\rho_j\}_{j \in [\ell]}$ and $\{\tau_j\}_{j \in [\ell]}$ be the triple sacrifice values corresponding to ℓ multiplication gates.

- HP samples a new PRF key k_{ver} and sends it to all the parties in $\mathcal{P}$.
- Sample a random values $\{r_j\}_{j \in [\ell]}$ and $\{s_j\}_{j \in [\ell]}$ using k_{ver} .
- Compute $\rho = \sum_{j=1}^{\ell} r_j \cdot \rho_j$.
- Compute $\langle \hat{\rho} \rangle = \sum_{j=1}^{\ell} r_j \cdot \langle \rho_j \rangle$ and $\langle \hat{\tau} \rangle = \sum_{j=1}^{\ell} s_j \cdot \langle \tau_j \rangle$.
- Reconstruct $\hat{\rho}, \hat{\tau}$ and the corresponding tags $t_{\hat{\rho}}$ and $t_{\hat{\tau}}$.
- Reconstruct $u = \hat{\rho} \cdot [\Delta] - t_{\hat{\rho}}$ and $v = \hat{\tau} \cdot [\Delta] - t_{\hat{\tau}}$.
- Verify $u = 0, v = 0, \hat{\rho} = \rho, \tau = \hat{\tau} = 0$ hold. If all checks pass, then parties output $\langle z \rangle$ corresponding to each multiplication gate. Otherwise output **abort**.

Fig. 11: Multiplying authenticated replicated shared values

Protocol $\Pi_{\text{Preproc}}^{\text{Castor}}$

Input: $\forall i \in [1, n]$, P_i and HP have the circuit C , common PRF keys $\{k_i, k_{\mathcal{P}}, k_{\text{all}}\}$ generated in the setup phase.

Output: $\forall i \in [1, n]$, P_i outputs preproc_i and HP outputs $\text{preproc}_{\text{HP}}$.

Protocol:

- *MAC key:* HP together with parties in $\mathcal{P}$ invokes Π_{RandRSS} (Fig. 8) to generate the share $[\Delta]$ of the global MAC key Δ .
- *Input gates:* For each input gate v with an input provider P_d , parties in $\mathcal{P}$ together with HP invoke Π_{RandARSS} (Fig. 9) to generate $\langle \delta_v \rangle$ such that δ_v is known to P_d .
- *Addition gates:* For each addition gate with inputs x and y , parties and the HP locally compute $\langle \delta_{x+y} \rangle = ([\delta_x] + [\delta_y], [t_x] + [t_y])$.
- *Multiplication gates:* For each multiplication gate $z = xy$, parties invoke Π_{RandARSS} (Fig. 9) to generate $\langle \delta_z \rangle$. Further, parties invoke $\Pi_{\text{mult-ARSS}}$ (Fig. 11) with inputs $\langle \delta_x \rangle, \langle \delta_y \rangle$ to obtain $\langle \delta_{xy} \rangle$.
- *Output gates:* For each output gate z , parties in $\mathcal{P}$ and HP invoke Π_{RandARSS} (Fig. 9) to generate $\langle \delta_z \rangle$.

Fig. 12: The preprocessing phase of Castor

4.3 The Online Phase:

Input gates. Let P_d be the party with input v . During the preprocessing phase, P_d has obtained the random mask, δ_v , for v in clear which is also $\langle \cdot \rangle$ -shared among all parties. P_d computes the masked value $m_v = v + \delta_v$ and sends it to HP, who then forwards m_v to all parties. All parties thus obtain $[\![v]\!] = (m_v, \langle \delta_v \rangle)$. Since the network is asynchronous, HP waits for messages from at most $n - t$ parties to avoid endless wait.

Protocol $\Pi_{[\![\cdot]\!]}^{\text{Castor}}$ -Sh

Input:

- Every input provider/dealer P_d , has input v and mask δ_v .
- $\forall i \in [1, n]$, P_i has the input gate g_{in} , $\langle \delta_v \rangle_i$ generated in Fig. 12 for g_{in} .

Output: $\forall i \in [1, n]$, P_i outputs $[\![v]\!]_i = (m_v, \langle \delta_v \rangle_i)$, where $m_v = v + \delta_v$.

Protocol:

- P_d computes $m_v = v + \delta_v$ and sends it to HP.
- Upon receiving m_v values corresponding to (at least) $n - t$ parties' inputs, HP forwards these to all parties.
- $\forall i \in [1, n]$, P_i outputs $[\![v]\!]_i = (m_v, \langle \delta_v \rangle_i)$.

Fig. 13: Online phase of input sharing of Castor

Addition gates. Due to the homomorphic property of the secret sharing scheme, each party locally obtains the sharing of the output wire of the addition gates.

Multiplication gates. Consider a multiplication gate with inputs x, y and output z such that parties hold (possibly incorrectly evaluated) $[\![x]\!], [\![y]\!]$. Moreover, parties already hold $\langle \delta_{xy} \rangle, \langle \delta_z \rangle$ from the preprocessing phase. $P_i \in \mathcal{P}$ computes the following.

$$[\![m_z]\!]_0 = m_x \cdot m_y - m_x \cdot [\delta_y]_0 - m_y \cdot [\delta_x]_0 + [\delta_{xy}]_0 + [\delta_z]_0$$

Each P_i sends $[\![m_z]\!]_0$ to HP. Simultaneously, HP computes $[\![m_z]\!]_i$ for each $i \in [1, n]$ as follows

$$[\![m_z]\!]_i = -m_x \cdot [\delta_y]_i - m_y \cdot [\delta_x]_i + [\delta_{xy}]_i + [\delta_z]_i$$

and sends $\tilde{m}_z = \sum_{i=1}^n [\![m_z]\!]_i$ to all parties. HP waits for $[\![m_z]\!]_0$ from a single party. Then all parties, including HP, compute $m_z = [\![m_z]\!]_0 + \tilde{m}_z$. Finally, parties output $[\![z]\!]$.

Protocol $\Pi_{\text{mult}}^{\text{Castor}}$

Input: $\forall i \in [1, n]$, P_i has the multiplication gate g_{mul} , $[\![x]\!]_i, [\![y]\!]_i$ and $\langle \delta_{xy} \rangle_i, \langle \delta_z \rangle_i$ generated in Fig. 12 for g_{mul} and the share of the global MAC key $[\Delta]$.

Output: $\forall i \in [1, n]$, P_i outputs $m_z = z + \delta_z$, where $z = x \cdot y$.

Protocol:

- $\forall i \in [1, n]$, P_i computes $[t_{m_z}]_0 = [\Delta]_0 \cdot (m_x \cdot m_y) - m_x \cdot [t_y]_0 - m_y \cdot [t_x]_0 + [t_{xy}]_0 + [t_z]_0$.
- $\forall i \in [1, n]$, HP computes $[t_{m_z}]_i = [\Delta]_i \cdot (m_x \cdot m_y) - m_x \cdot [t_y]_i - m_y \cdot [t_x]_i + [t_{xy}]_i + [t_z]_i$.
- $\forall i \in [1, n]$, P_i computes $[m_z]_0 = m_x \cdot m_y - m_x \cdot [\delta_y]_0 - m_y \cdot [\delta_x]_0 + [\delta_{xy}]_0 + [\delta_z]_0$ and sends it to HP.
- HP computes $\tilde{m}_z = \sum_{i=1}^n [m_z]_i$, where $[m_z]_i = -m_x \cdot [\delta_y]_i - m_y \cdot [\delta_x]_i + [\delta_{xy}]_i + [\delta_z]_i$, $\forall i \in [1, n]$.
- HP waits to receive $[m_z]_0$ from a single party. All parties including HP reconstruct $m_z = [m_z]_0 + \tilde{m}_z$.
- $\forall i \in [1, n]$, P_i outputs $([z]_i, [t_{m_z}]_i)$, where $[z]_i = (m_z, \langle \delta_z \rangle_i)$.

Fig. 14: Online phase of multiplication of Castor

Verification. The verification is similar to $\Pi_{\text{verify}}^{\text{Pollux}}$ from Section 3. However, in this case HP also prepares the shares and it misses only one component. All the parties compute that component and send it to HP. HP waits to receive the missing share from one party, upon receiving it reconstructs and checks if the reconstructed value is 0. We provide the details in Fig. 15.

Protocol $\Pi_{\text{verify}}^{\text{Castor}}$

Input: $\forall i \in [1, n]$, P_i has the reconstructed masked values $\{m_{z_1}, m_{z_2}, \dots, m_{z_\ell}\}$, and $[t_{m_{z_j}}]_0$ as the share of MAC on m_{z_j} , for $j \in [1, \ell]$. P_i also has the share of the global MAC key $[\Delta]_0$.

Output: All parties and HP output 1 if the verification passes, else 0.

Protocol:

- HP samples a new PRF key k_{ver} and sends it to all the parties in $\mathcal{P}$.
- $\forall i \in [1, n]$, P_i computes $[\omega_{z_j}]_0 = [t_{m_{z_j}}]_0 - m_{z_j} \cdot [\Delta]_0$.
- $\forall i \in [1, n]$, P_i samples $\rho = (\rho_1, \dots, \rho_\ell) \leftarrow \mathbb{F}^\ell$ using k_{ver} .
- $\forall i \in [1, n]$, P_i computes $[\omega_z]_0 = \sum_{j \in [\ell]} \rho_j \cdot [\omega_{z_j}]_0$ and sends to HP.
- HP computes $[\omega_{z_j}]_i = [t_{m_{z_j}}]_i - m_{z_j} \cdot [\Delta]_i$, $\forall j \in [1, \ell]$.
- $\forall i \in [1, n]$, HP computes $[\omega_z]_i = \sum_{j \in [\ell]} \rho_j \cdot [\omega_{z_j}]_i$.
- HP waits to receive $[\omega_z]_0$ from a single party. Then it reconstructs $\omega_z = \sum_{i=0}^n [\omega_z]_i$ and checks if $\omega_z = 0$. If the check fails, then HP sends 0 to all, else it sends 1.

Fig. 15: Amortized MAC-verification of Castor

Output reconstruction. Let z denote an output that is $[\![\cdot]\!]$ -shared. If the output of the MAC verification is 1, then HP sends $\tilde{\delta}_z = \sum_{i=1}^n [\delta_z]_i$ to all parties. Each party in $\mathcal{P}$ waits to receive $\tilde{\delta}_z$ from HP and then locally computes the final output as $z = m_z - ([\delta_z]_0 + \tilde{\delta}_z)$.

Protocol $\Pi_{\text{Online}}^{\text{Castor}}$ **Input:**

- For every input provider/dealer P_d , it has input v .
- $\forall i \in [1, n]$, P_i and HP have the circuit C , the preprocessing data generated in Fig. 12 and common PRF keys $\{k_i, k_{\mathcal{P}}, k_{\text{all}}\}$ generated in the setup phase.

Output: $\forall i \in [1, n]$, P_i outputs $C(x_1, x_2, \dots, x_n)$ or $\perp$.

Protocol:

- *Input gates:* For every input gate of C , parties execute $\Pi_{[\![\cdot]\!]-\text{Sh}}^{\text{Castor}}$ (Fig. 13).
- *Addition gates:* For every addition gate of C with inputs x, y and output z , parties and HP locally set $m_z = m_x + m_y$.
- *Multiplication gates:* For every multiplication gate of C , parties execute $\Pi_{\text{mult}}^{\text{Castor}}$ (Fig. 14).

- *Amortized verification:* At the end of all the multiplication gates, all parties and HP execute $\Pi_{\text{verify}}^{\text{Castor}}$ (Fig. 15). If parties output 1, then **continue**, else output $\perp$.
- *Output gates:*
 - If the output of *Verification* is 1, then HP computes $\tilde{\delta}_z = \sum_{i=1}^n [\delta_z]_i$ for all output gates and sends them to all parties.
 - If the output of *Verification* is 0, it sends $\perp$ to all parties.

Fig. 16: The online phase of Castor

Security. We defer a detailed discussion on the security of Castor to Appendix F.2.

4.4 Necessity of Oblivious Transfer

In this section, we show the necessity of OT for HP-aided AMPC with dishonest majority.

Theorem 1. *A $(n+1)$ -party HP-aided AMPC protocol among parties $\{P_1, \dots, P_n\}$ and HP, where either at most $n-1$ parties are maliciously corrupt or HP is semi-honest implies a 2-party synchronous OT protocol which is secure against semi-honest sender and malicious receiver.*

Proof sketch. To prove the claim, we consider the following function $f((m_0, m_1), \mathbf{b}, \perp, \dots, \perp) = (\perp, m_{\mathbf{b}}, \perp, \dots, \perp)$, where HP holds inputs (m_0, m_1) , P_1 holds a choice bit $\mathbf{b}$, and the remaining parties $P_2, \dots, P_n$ have no inputs. The output of the function is $m_{\mathbf{b}}$ delivered only to P_1 . Suppose π_f be a HP-aided AMPC protocol that securely realizes the above functionality. First, we consider an execution where all parties behave honestly and follow the protocol, but the network adversary (due to asynchrony) delays all messages to and from $P_2, \dots, P_n$. Since there is no malicious behavior, P_1 must still receive the correct output $m_{\mathbf{b}}$.

Using π_f , we construct a synchronous 2-party OT protocol π_{OT} between a sender S (with inputs (m'_0, m'_1)) and a receiver R (with choice bit c). S emulates the role of HP in π_f , using $(m_0, m_1) = (m'_0, m'_1)$, and sends messages to R whenever π_f specifies sending messages to P_1 (ignoring all other parties). Similarly, R emulates the role of P_1 with input $\mathbf{b} = c$, and only communicates with S when π_f dictates communication with HP, ignoring all other parties.

The correctness of π_{OT} follows directly from the correctness of π_f under delayed communication from the other parties. Security against a semi-honest S in π_{OT} follows from the security of π_f against a semi-honest HP. Security against a malicious R follows from the security of π_f against a malicious P_1 , even when the other parties are inactive. Hence, this reduction demonstrates the necessity of oblivious transfer for asynchronous HP-aided MPC with dishonest majority. Due to space constraints, the full formal proof is provided in Appendix D.

5 Implementation and Experimental Results

We describe prototype implementations of Pollux and Castor, which we use for benchmarking and comparison with Asterisk [29], the state-of-the-art helper-aided protocol in synchronous settings.

Environment. Benchmarks are carried out on a WAN network simulated on a local server, equipped with a 4.5GHz, 24 cores, 48 threads, AMD Ryzen™ Threadripper PRO 5965WX processor having 256GB RAM. Each party P_i in $\mathcal{P}$, as well as HP, is executed as a separate process on a single server. Additionally, we utilize 64 threads within HP to model an asynchronous network, leveraging its computational capacity to support multithreading. HP uses a dedicated thread for communicating with each party P_i , allowing parallel message exchange. In each round, HP proceeds once it has successfully received messages from any $n-t$ parties (i.e., once $n-t$ threads have completed their reception), without waiting for the remaining messages. We consider a bandwidth of 100 Mbps and 50 ms latency and use the Linux `tc` command from the network emulation package `netem` to emulate a WAN network for interprocess communication. Both Pollux and Castor are benchmarked in WAN, as asynchronous networks are typically more suited to WAN environments. Each of the test suites is run ten times without interruption, and the average runtime is reported.

Implementation of Pollux and Castor. We implement our protocols⁷ in C++17, leveraging the codebase of Asterisk [29] and EMP toolkit [40]. For the OT-based primitives required during the preprocessing phase of Castor, we employ maliciously secure Ferret-COT [41] from the same EMP toolkit [40]. We set the computational security parameter $\kappa = 128$ and ensure a statistical security of at least 2^{-40} . We use the NTL library [37] for all $\mathbb{Z}_p$ operations, where p is a 64-bit prime.

Instantiation of Asterisk. We compare the online phase of our protocols with Asterisk [29] by introducing an artificial delay of 200 ms for a subset of parties. To facilitate this, we leverage the publicly available implementation of Asterisk. The delay is introduced by modifying the selected parties to wait 200 ms before sending any message in each round. This setup simulates a network adversary who delays a subset of parties by disrupting their communication channels.

5.1 Performance benchmarks

We evaluate Pollux and Castor on synthetic circuits with a total of 1 million multiplication gates. We consider circuits with depth $d = 10$ and $d = 100$, containing 10^5 and 10^4 multiplication gates per level, respectively. Additionally, we vary the number of parties n from 5 to 30. The corresponding runtimes and communication overheads for Pollux and Castor protocols are reported in Table 1.

d	n	Preprocessing						Online		
		comm. (MB)			runtime (sec)			comm. (MB)	runtime (sec)	
		Pollux	Castor		Pollux	Castor		Pollux/ Castor	Pollux	Castor
			OLE	Remaining		OLE	Remaining			
10	5	96.13	60241.45	199.13	24.85	5186.26	45.95	76.29	25.22	10.6
	10	192.26	120482.91	351.72	50.81	10361.04	94.83	152.59	33.7	19.83
	20	352.48	240965.81	656.89	125.56	20446.41	255.69	305.18	54.22	41.61
	30	512.7	361448.72	962.07	229.09	30502.42	485.9	457.77	76.27	65.4
100	5	92.01	59353.04	198.44	23.9	5107.17	44.58	76.3	20.78	12.39
	10	184.02	118706.09	351.03	48.97	10037.59	94.72	152.6	26.56	19.24
	20	337.37	237412.17	656.21	120.95	20127.52	253.67	305.19	40.38	39.93
	30	490.72	356118.26	961.38	220.32	30061.92	481.84	457.79	60.72	65.73

Table 1: The preprocessing and online overheads of Pollux and Castor for circuits with 10^6 multiplication gates, for varying depth ($d \in \{10, 100\}$) and varying numbers of parties ($n \in \{5, 10, 20, 30\}$).

The minor differences observed in preprocessing overheads with varying d are due to the change in the number of input gates, from 10^5 to 10^4 , when the circuit depth increases from 10 to 100. Furthermore, the significant preprocessing overhead in Castor is primarily attributed to the mandatory OT operations required for OLE during this phase. As shown in Table 1, excluding the cost of OLE, the overhead remains minimal. To manage memory usage and avoid RAM overflow, we process OLEs in chunks of 50000. This chunk size can be increased to further reduce the overall overhead of the OLE.

In the online phase, although the communication overhead is theoretically the same regardless of d or the underlying protocol (Pollux or Castor), we observe minor variations in the experiment when varying d .

⁷ Our implementation is publicly available at: <https://github.com/cris-coders-iisc/Asynchronous-HP-aided-MPC>. Our code is developed for benchmarking and is not optimized for industry-grade use.

This discrepancy arises because, in the implementation, each party $P_i \in \mathcal{P}$ sends an additional field element at each level to indicate the number of bytes it will transmit to HP to process the multiplication gates at that level. Furthermore, although Pollux and Castor incur similar communication costs, the online runtime for Pollux is comparatively higher due to the heavier local computation performed by HP, specifically, the Lagrange interpolation, for reconstructing $\mathbf{m}_z$ and the additional round required in Pollux.

5.2 Effect of asynchronous network

Asynchronous protocols typically require more communication than synchronous ones. However, given the same communication, asynchronous protocols often achieve better performance because they do not wait for all parties' messages, while synchronous protocols do. This is evident from the comparison of the online phases of Pollux/Castor and Asterisk: Asterisk waits for all parties, Pollux proceeds after receiving from half the parties, and Castor continues with just one party's message. Moreover, Castor requires one round of communication per multiplication, compared to two rounds in both Asterisk and Pollux, further accelerating its online phase.

We compare the performance of the online phase of our protocols with that of Asterisk [29], by introducing an artificial delay of 200 ms for a subset of parties. We assume that in a synchronous network the round delay is greater than 200 ms. This implies that the recipient parties will wait for delayed messages without flagging the sender as malicious. The effect of delay is not observable during the preprocessing phase in Pollux and Asterisk, as the communication occurs only from HP to the other parties. Although Castor involves communication from the parties to HP during preprocessing, due to its higher preprocessing cost, we limit our comparison to the online phase only.

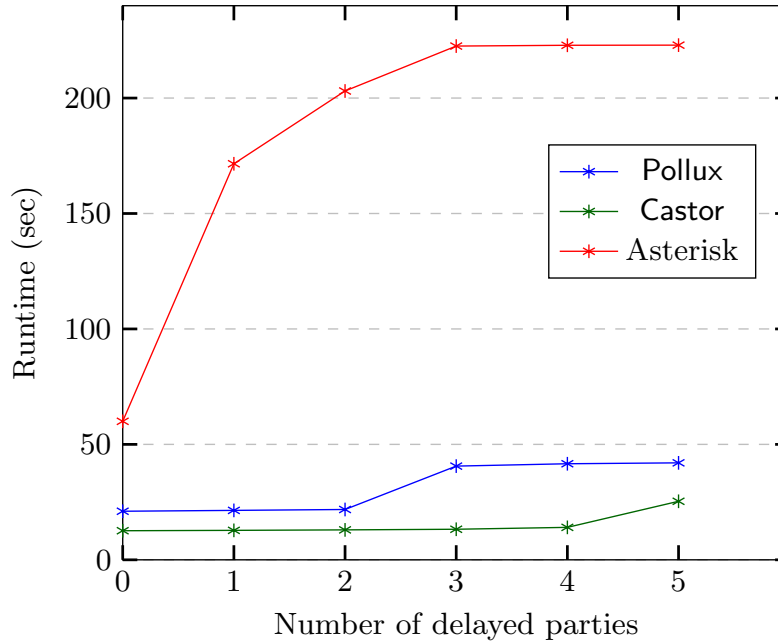


Fig. 17: Effect of asynchronous network delays on the online running time of Pollux, Castor, and Asterisk for $n = 5$ parties and for circuits with 10^6 multiplication gates of depth $d = 100$. Delays of 200 ms were incrementally introduced for 0, 1, 2, 3, 4, and 5 parties. The performance degradation of Asterisk due to delays is significantly larger than that of Pollux and Castor.

For this demonstration, we fix the number of parties to $n = 5$ and introduce delays incrementally for 1, 2, 3, 4 and 5 parties, where delayed parties wait 200 ms before sending any message in each round. We

measure the average runtime experienced by the non-delayed parties⁸. As shown in Figure 17, for Pollux, the runtime remains unaffected as long as fewer than $n/2$ parties are delayed. When the number of delayed parties equals or exceeds $n/2$, the impact of the delay becomes visible in the runtime. In contrast, for Castor, the runtime remains stable even when up to $n - 1$ parties are delayed, and the delay becomes apparent only when all n parties are delayed. On the other hand, Asterisk exhibits noticeable delay as soon as even a single party is delayed, since it operates in a synchronous network setting where all parties must wait for messages from every other party before proceeding. Also note that in the online phase of Asterisk, the reconstruction of $\mathbf{m}_z$ is performed by the computing parties themselves, not via HP. Thus, a multithreaded HP would not substantially speed up the online phase of Asterisk.

6 Conclusion and Open Questions

We presented two helper-aided AMPC protocols, Pollux and Castor. To the best of our knowledge, this is the first work that proposes practical asynchronous solutions that overcome the traditional threshold barriers. We demonstrated their practicality through extensive implementation and comparison with the state-of-the-art synchronous MPC protocol. Notably, we showed that the performance of a synchronous protocol degrades significantly in the presence of a slow party, while our constructions remain robust. Pollux tolerates up to $(n - 1)/2$ corrupt or slow parties, whereas Castor tolerates $n - 1$ such parties, albeit with a more expensive preprocessing phase. This suggests a compelling model where preprocessing could be performed synchronously (with longer per-round time bounds), followed by an efficient asynchronous online phase—a direction we leave for future work. As is well-known, AMPC protocols are limited to incorporating inputs from at most $n - t$ parties. Our constructions also inherit this limitation. One direction to overcome this would be to allow a single synchronous round during the input sharing phase. We leave designing a protocol that accepts inputs from all the parties without a synchronous round as an interesting open question.

Another interesting open direction is to reduce the number of calls to HP. More concretely, our protocols invoke the HP $O(d)$ times, compared to only four invocations in Asterisk [29]. This increased reliance on the HP in the asynchronous setting arises from the following fundamental challenge: Consider an honest party waiting for messages from others. In the synchronous setting, if it does not receive messages within a predefined time, it can abort. However, in the asynchronous setting, this can be handled only by either (i) using the helper for whom the honest party can afford to wait for or (ii) relying on other honest parties, which would require an honest majority. Since Castor is designed for the dishonest majority setting, an honest party may not receive any message; making it necessary to rely on approach (i). However, in the case of Pollux which tolerates an honest majority, both approaches (i) and (ii) could be considered. In this paper, we prefer approach (i) over approach (ii), as using approach (ii) would incur significantly higher communication cost (specifically $O(n^2)$ as opposed to $O(n)$ in approach (i) if we rely on $O(n)$ parties to act as P_{king}). Reducing the number of invocations of the HP while maintaining comparable efficiency as Pollux is highly non-trivial. We leave this as an interesting direction of future research.

Acknowledgments

This project received funding from the ERC under the EU’s research and innovation programs Horizon 2020 (PSOTI/850990) and Horizon Europe (PRIVTOOLS/101124778). It was co-funded by the DFG within SFB 1119 CROSSING/236615297. Divya Ravi was supported by the gravitation project Challenges in Cyber Security (CiCS) with file number 024.006.037 which is financed by the Dutch Research Council (NWO). Banashri Karmakar was financially supported by the TCS Foundation through the TCS Research Scholar Program. This work is partially supported by a Stellar Development Foundation Academic Research Award. Arpita Patra would like to acknowledge financial support from Kotak-IISc AI-ML Centre, K-Tech CoE for Cyber Security, J.P Morgan Chase Faculty Award 2023, Google Privacy Research Award 2023.

⁸ When all 5 parties are delayed, the average runtime considering all 5 parties is reported.

References

1. Ittai Abraham, Gilad Asharov, Shravani Patil, and Arpita Patra. Perfect asynchronous mpc with linear communication overhead. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 280–309. Springer, 2024.
2. Gilad Asharov, Tucker Hybinette Balch, Antigoni Polychroniadou, and Manuela Veloso. Privacy-preserving dark pools. In *Proceedings of the 19th International Conference on Autonomous Agents and MultiAgent Systems*, pages 1747–1749, 2020.
3. Akhil Bandrupalli, Xiaoyu Ji, Aniket Kate, Chen-Da Liu-Zhang, and Yifan Song. Concretely efficient asynchronous mpc from lightweight cryptography. *Cryptology ePrint Archive*, 2024.
4. Zuzana Beerliová-Trubíniová, Martin Hirt, and Jesper Buus Nielsen. On the theoretical gap between synchronous and asynchronous mpc protocols. In *Proceedings of the 29th ACM SIGACT-SIGOPS symposium on Principles of distributed computing*, pages 211–218, 2010.
5. Aner Ben-Efraim, Michael Nielsen, and Eran Omri. Turbospeedz: Double your online spdz! improving spdz using function dependent preprocessing. In *Applied Cryptography and Network Security: 17th International Conference, ACNS 2019, Bogota, Colombia, June 5–7, 2019, Proceedings 17*, pages 530–549. Springer, 2019.
6. Michael Ben-Or, Boaz Kelmer, and Tal Rabin. Asynchronous secure computations with optimal resilience. In *Proceedings of the thirteenth annual ACM symposium on Principles of distributed computing*, pages 183–192, 1994.
7. Erica Blum, Jonathan Katz, Chen-Da Liu-Zhang, and Julian Loss. Asynchronous byzantine agreement with subquadratic communication. In *Theory of Cryptography: 18th International Conference, TCC 2020, Durham, NC, USA, November 16–19, 2020, Proceedings, Part I 18*, pages 353–380. Springer, 2020.
8. Ran Canetti. Universally composable security: A new paradigm for cryptographic protocols. In *FOCS*, 2001.
9. Ran Canetti, Asaf Cohen, and Yehuda Lindell. A simpler variant of universally composable security for standard multiparty computation. In *Advances in Cryptology–CRYPTO 2015: 35th Annual Cryptology Conference, Santa Barbara, CA, USA, August 16–20, 2015, Proceedings, Part II 35*, pages 3–22. Springer, 2015.
10. John Cartledge, Nigel P Smart, and Younes Talibi Alaoui. Mpc joins the dark side. In *Proceedings of the 2019 ACM Asia Conference on Computer and Communications Security*, pages 148–159, 2019.
11. Harsh Chaudhari, Ashish Choudhury, Arpita Patra, and Ajith Suresh. Astra: High throughput 3pc over rings with application to secure prediction. In *Proceedings of the 2019 ACM SIGSAC Conference on Cloud Computing Security Workshop*, pages 81–92, 2019.
12. Annick Chopard, Martin Hirt, and Chen-Da Liu-Zhang. On communication-efficient asynchronous mpc with adaptive security. In *Theory of Cryptography: 19th International Conference, TCC 2021, Raleigh, NC, USA, November 8–11, 2021, Proceedings, Part II 19*, pages 35–65. Springer, 2021.
13. Ashish Choudhury and Arpita Patra. Optimally resilient asynchronous mpc with linear communication complexity. In *Proceedings of the 16th International Conference on Distributed Computing and Networking*, pages 1–10, 2015.
14. Ashish Choudhury and Arpita Patra. On the communication efficiency of statistically secure asynchronous mpc with optimal resilience. *Journal of Cryptology*, 36(2):13, 2023.
15. Richard Cleve. Limits on the security of coin flips when half the processors are faulty (extended abstract). In Juris Hartmanis, editor, *Proceedings of the 18th Annual ACM Symposium on Theory of Computing, May 28–30, 1986, Berkeley, California, USA*, pages 364–369. ACM, 1986.
16. Ran Cohen. Asynchronous secure multiparty computation in constant time. In *Public-Key Cryptography–PKC 2016*, pages 183–207. Springer, 2016.
17. Ronald Cramer, Ivan Damgård, Daniel Escudero, Peter Scholl, and Chaoping Xing. Spdz2k: Efficient mpc mod 2k for dishonest majority. In *Annual International Cryptology Conference. Springer*, pages 769–798, 2018.
18. Ronald Cramer, Ivan Damgård, and Jesper B Nielsen. Multiparty computation from threshold homomorphic encryption. In *International conference on the theory and applications of cryptographic techniques*, pages 280–300. Springer, 2001.
19. Ivan Damgård, Martin Geisler, Mikkel Kroigaard, and Jesper Buus Nielsen. Asynchronous multiparty computation: Theory and implementation. In *International workshop on public key cryptography*, pages 160–179. Springer, 2009.
20. Ivan Damgård and Jesper Buus Nielsen. Scalable and unconditionally secure multiparty computation. In *Annual International Cryptology Conference*, pages 572–590. Springer, 2007.
21. Ivan Damgård, Valerio Pastro, Nigel Smart, and Sarah Zakarias. Multiparty computation from somewhat homomorphic encryption. In *Annual Cryptology Conference*, pages 643–662. Springer, 2012.

22. Michael J. Fischer, Nancy A. Lynch, and Mike Paterson. Impossibility of distributed consensus with one faulty process. *J. ACM*, 32(2):374–382, 1985.
23. Mariana Gama, Emad Heydari Beni, Emmanuela Orsini, Nigel P Smart, and Oliver Zajonc. Mpc with delayed parties over star-like networks. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 172–203. Springer, 2023.
24. Vipul Goyal, Chen-Da Liu-Zhang, and Yifan Song. Towards achieving asynchronous mpc with linear communication and optimal resilience. In *Annual International Cryptology Conference*, pages 170–206. Springer, 2024.
25. Martin Hirt, Jesper Buus Nielsen, and Bartosz Przydatek. Cryptographic asynchronous multi-party computation with optimal resilience. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 322–340. Springer, 2005.
26. Martin Hirt, Jesper Buus Nielsen, and Bartosz Przydatek. Asynchronous multi-party computation with quadratic communication. In *International Colloquium on Automata, Languages, and Programming*, pages 473–485. Springer, 2008.
27. Yuval Ishai, Arpita Patra, Sikhar Patranabis, Divya Ravi, and Akshayaram Srinivasan. Fully-secure mpc with minimal trust. In *Theory of Cryptography Conference*, pages 470–501. Springer, 2022.
28. Xiaoyu Ji, Junru Li, and Yifan Song. Linear-communication asynchronous complete secret sharing with optimal resilience. In *Annual International Cryptology Conference*, pages 418–453. Springer, 2024.
29. Banashri Karmakar, Nishat Koti, Arpita Patra, Sikhar Patranabis, Protik Paul, and Divya Ravi. Asterisk: Super-fast mpc with a friend. In *2024 IEEE Symposium on Security and Privacy (SP)*, pages 542–560. IEEE, 2024.
30. Marcel Keller, Emmanuela Orsini, and Peter Scholl. Mascot: faster malicious arithmetic secure computation with oblivious transfer. In *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, pages 830–842, 2016.
31. Marcel Keller, Valerio Pastro, and Dragos Rotaru. Overdrive: Making spdz great again. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 158–189. Springer, 2018.
32. Atsuki Momose. Practical asynchronous mpc from lightweight cryptography. *Cryptology ePrint Archive*, 2024.
33. Arpita Patra, Ashish Choudhary, and C Pandu Rangan. Efficient statistical asynchronous verifiable secret sharing with optimal resilience. In *International Conference on Information Theoretic Security*, pages 74–92. Springer, 2009.
34. Arpita Patra, Ashish Choudhury, and C Pandu Rangan. Efficient asynchronous multiparty computation with optimal resilience. *Cryptology ePrint Archive*, 2008.
35. Arpita Patra, Thomas Schneider, Ajith Suresh, and Hossein Yalame. {ABY2. 0}: Improved {Mixed-Protocol} secure {Two-Party} computation. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 2165–2182, 2021.
36. Antigoni Polychroniadou, Gilad Asharov, Benjamin Diamond, Tucker Balch, Hans Buehler, Richard Hua, Suwen Gu, Greg Gimler, and Manuela Veloso. Prime match: A {Privacy-Preserving} inventory matching system. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 6417–6434, 2023.
37. Victor Shoup. NTL: A Library for doing Number Theory. <https://libnt1.org/>, 2021.
38. Yuan Su, Yuan Lu, Jiliang Li, Yuyi Wang, Chengyi Dong, and Qiang Tang. Dumbo-mpc: Efficient fully asynchronous mpc with optimal resilience. *Cryptology ePrint Archive*, 2024.
39. Sam Toueg. Randomized byzantine agreements. In Tiko Kameda, Jayadev Misra, Joseph G. Peters, and Nicola Santoro, editors, *Proceedings of the Third Annual ACM Symposium on Principles of Distributed Computing, Vancouver, B. C., Canada, August 27-29, 1984*, pages 163–178. ACM, 1984.
40. Xiao Wang, Alex J. Malozemoff, and Jonathan Katz. EMP-toolkit: Efficient MultiParty computation toolkit. <https://github.com/emp-toolkit>, 2016.
41. Kang Yang, Chenkai Weng, Xiao Lan, Jiang Zhang, and Xiao Wang. Ferret: Fast extension for correlated ot with small communication. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, 2020.
42. Andrew C Yao. Protocols for secure computations. In *23rd annual symposium on foundations of computer science (sfcs 1982)*, pages 160–164. IEEE, 1982.

A (Recap) Impossibility of Asynchronous Protocols for $n = 3t$

The classic impossibility result of asynchronous byzantine agreement when $n \leq 3t$ [39] extends to even security with abort protocols and even in the presence of setup. For completeness, we recall the impossibility here. Suppose there exists an asynchronous byzantine agreement protocol when $n = 3t$. Consider a partition of the set of parties into sets A, B and C each of size t . Consider a sequence of scenarios:

- Scenario 1: Consider an all honest execution where all parties have input 0. Suppose all the messages outgoing from C are delayed. Since parties in A and B cannot distinguish whether C is corrupt or simply slow, to make sure validity holds, they must output 0. Note that this holds even for security with abort as this is an all honest execution where everyone behaves honestly, except that the messages from C are delayed.
- Scenario 2: Consider an all honest execution where parties in A and B have input 0 and parties in C have input 1. Suppose the messages from C are arbitrarily delayed. Then, since the views of parties in A and B are identically distributed as in Scenario 1, they output 0.
- Scenario 3: Consider the symmetric case of an execution where parties in B and C have input 1 and messages from parties in A are arbitrarily delayed. By similar arguments as above, one concludes that parties in B and C output 1.
- Scenario 4: Consider an execution where parties in A have input 0, parties in C have input 1 and the t parties in B are maliciously corrupt. Parties in B act as per input 0 towards parties in A and pretend they received nothing from parties in C . Similarly, parties in B act as per input 1 towards parties in C and pretend they received nothing from parties in A . Further, the messages between parties A and C are arbitrarily delayed (more specifically they are delayed by $> T$ timesteps, where T is larger than the time that parties in A and C wait before outputting a value in Scenarios 2 & 3 respectively.).

In Scenario 4, one can observe that the views of parties in A are identically distributed to their views in Scenario 2; therefore they must output 0. On the other hand, the views of parties in C is identically distributed to their views in Scenario 3; therefore they must output 1. This violates consistency of the byzantine agreement protocol, which is a contradiction.

We note that since the inferences here only relied on output being computed in all honest executions, they must hold also for security with abort.

B Additional Preliminaries

B.1 Security Model

We prove the security of our protocols in the standard real world/ideal world paradigm where the security of a protocol is analyzed by comparing what an adversary can do in the real execution of the protocol, to what it can do in an ideal execution which is secure by definition (in the presence of an incorruptible trusted party).

The real world An n -party protocol π with parties $\mathcal{P} = \{P_1, \dots, P_n\}$ is an n -tuple of probabilistic polynomial-time (PPT) interactive Turing machines (ITMs). We let $\mathcal{A}$ denote a special PPT ITM that represents the adversary and is initialized with an input that contains the identities of the corrupt parties, their respective private inputs, and an auxiliary input. During the execution of the protocol, maliciously corrupt parties receive arbitrary instructions from adversary $\mathcal{A}$, while honest parties and semi-honestly corrupt parties faithfully follow the instructions of the protocol. We consider the adversary $\mathcal{A}$ to be rushing, that is, during every round the adversary can see the messages the honest parties sent before producing messages from actively corrupt parties. At the end of the protocol execution, honest parties produce output, the corrupt parties produce no output, and the adversary outputs an arbitrary function of its view. The view of a party during the execution consists of its input, random coins, and the messages it sees during the execution.

Definition 1 (Real-world execution). Let π be an n -party protocol amongst $\{P_1, \dots, P_n\}$ computing an n -party function $f : (\{0, 1\}^*)^n \rightarrow (\{0, 1\}^*)^n$ and let $C \subset [1, n] \cup \{HP\}$ denote the set of indices of the corrupted parties. The execution of π under $(\mathcal{A}, C)$ in the real world, on input vector $\vec{x} = (x_1, \dots, x_n)$, auxiliary input $\mathbf{aux}$ and security parameter κ , denoted $\mathbf{real}_{\pi, C, \mathcal{A}(\mathbf{aux})}(\vec{x}, \kappa)$, is defined as the output vector of $P_1, \dots, P_n$ and $\mathcal{A}(\mathbf{aux})$ resulting from the protocol interaction.

The ideal world We describe ideal world execution with the fairness security guarantee.

Definition 2 (Ideal Computation). Let $f : (\{0, 1\}^*)^n \rightarrow (\{0, 1\}^*)^n$ be an n -party function and let $C \subset [1, n] \cup \{HP\}$ denote the set of all corrupt parties. Then, the joint ideal execution of f under $(\mathcal{S}, C)$ on input vector $\vec{x} = (x_1, \dots, x_n)$, auxiliary input $\mathbf{aux}$ to $\mathcal{S}$ and security parameter κ , denoted $\mathbf{ideal}_{f,C,\mathcal{S}(\mathbf{aux})}(\vec{x}, \kappa)$, is defined as the output vector of $P_1, \dots, P_n$ and $\mathcal{S}$ resulting from the following ideal process.

1. Parties send inputs to trusted party: An honest party P_i sends its input x_i to the trusted party. The simulator $\mathcal{S}$ may send to the trusted party arbitrary inputs for the actively corrupt parties. Let x'_i be the value actually sent as the input of party P_i .
2. Simulator speaks to the trusted party: The simulator $\mathcal{S}$ sends to the trusted party a set H of (at most) t parties whose inputs will not be considered in the computation.
3. Trusted party speaks to simulator: The trusted party computes $(y_1, \dots, y_n) = f(x'_1, \dots, x'_n)$ where $x'_i = 0$ if $P_i \in H$. The trusted party sends **ready** to $\mathcal{S}$.
4. Simulator $\mathcal{S}$ responds to trusted party: The simulator can send **abort** to the trusted party.
5. Trusted party answers parties:
 1. If the trusted party received **abort** from the simulator $\mathcal{S}$, it sets the abort message $\perp$.
 2. Otherwise, it sends y_j to every P_j , for all $j \in [1, n]$.
6. Outputs: Honest parties always output the message received from the trusted party, while corrupt parties output nothing. The simulator $\mathcal{S}$ outputs an arbitrary function of the initial inputs $\{x_i\}_{i \in C}$, the messages received by the corrupt parties from the trusted party and its auxiliary input.

In this work, we consider the non-colluding security notion of [27], which is described below.

Non-colluding security Informally, protocols with this notion of security are secure against a non-colluding adversary that corrupts *either* any t -size subset of the n parties $\{P_1, \dots, P_n\}$ maliciously *or* the HP passively (i.e. in a semi-honest manner). The formal definition appears below.

Definition 3 (Non-colluding security). Let $f : (\{0, 1\}^*)^n \rightarrow (\{0, 1\}^*)^n$ be an n -party function. A protocol Π securely computes the function f in the non-colluding model with fairness security if for any adversary $\mathcal{A}$, there exists a simulator $\mathcal{S}$ such that for every $\mathcal{C}$ where either (a) $\mathcal{C} \subset [1, n]$ and $\|\mathcal{C}\| = t$ malicious corruptions or (b) $\mathcal{C} = \text{HP}$ semi-honest corruption, we have

$$\{\mathbf{real}_{\Pi, \mathcal{C}, \mathcal{A}(\mathbf{aux})}(\vec{x}, \kappa)\}_{\vec{x} \in (\{0, 1\}^*)^n, \kappa \in \mathbb{N}} \equiv \{\mathbf{ideal}_{f, \mathcal{C}, \mathcal{S}(\mathbf{aux})}(\vec{x}, \kappa)\}_{\vec{x} \in (\{0, 1\}^*)^n, \kappa \in \mathbb{N}}.$$

Note that the corruption is non-simultaneous. Therefore we need the above indistinguishability to hold in both the corruption cases. Finally, a protocol achieves computational security, if the above distributions are computationally close in the presence of the parties, $\mathcal{A}$, $\mathcal{S}$ that are PPT. A protocol achieves statistical (resp. perfect) security if the distributions are statistically close (resp. identical).

B.2 Details of Network Model

Here, in the ideal world, the parties, adversary and ideal functionality are connected in a star configuration, where all communication is performed via an additional router machine that is under the adversary's control. Each entity has one outgoing channel and one incoming channel to and from the router. When P_i sends a message to P_j , it sends it to the router, and the message is stored by the router. The router delivers general information about the message to the adversary. Specifically, the adversary is allowed to know the type of message and its length, but not its content. When the adversary allows the delivery of the message, the router delivers the message to P_j . To model eventual delivery, we quantify over only those adversaries that eventually deliver all messages. In summary, in an execution with an ideal functionality, communication between the parties and this functionality is done via the router machine and is subject to (finite) delivery delays imposed by the adversary. When the functionality gives an output to party P_j , this is performed via the router, and the simulator is notified. Thus, if the adversary, delays the delivery of the output of some

party P_j , we do not explicitly mention that in the functionality, yet it is captured by the model. Similarly, we eliminate details regarding the router from the simulation, however, the scheduling of messages is performed via the router to ensure indistinguishable views in the real and ideal worlds.

B.3 Shared key setup

Note that $\{k_i\}_{i \in [1, n]}$ and k_{all} can be sampled by HP and directly distributed to the respective parties. However, generating k_P is more involved, as it must ensure agreement among all parties even in the presence of potentially malicious participants. This can be achieved using a single round of synchronous communication, leveraging the existing PKI setup for secure channels and NIZK proofs. Since this setup is a one-time cost, its overhead can be amortized. We omit the details due to space constraints.

C Proof of Necessity of Input Deprivation

Claim. An HP-aided asynchronous protocol achieving security with abort without input deprivation is impossible.

Proof. Assume that π_f is an HP-aided asynchronous MPC protocol that securely computes $f(x_1, \dots, x_n)$ (where x_i denotes the input of P_i) and does not suffer from input deprivation. Consider an execution where everyone behaves honestly, but the outgoing messages of P_1 are arbitrarily delayed i.e. none of the parties receives a message from P_1 before they terminate. Note that the other parties cannot afford to wait for P_1 's messages before terminating as they cannot distinguish whether P_1 is a slow honest sender or a corrupt party who did not send a message. By the correctness of the protocol π_f , we can conclude that parties will receive the correct output $f(x_1, \dots, x_n)$. Note that this must be the case even for security with abort protocols as the scenario considered is an all-honest execution. Furthermore, since input deprivation is not allowed, this output must be computed on the honest input of P_1 , namely x_1 . However, we note that the entire computation during the protocol must have been independent of P_1 's input, as none of the messages of P_1 were delivered to the other participants. This means that the protocol is vulnerable to a residual attack, where any party can locally obtain multiple evaluations of the function with different choices of P_1 's input. This contradicts security of π_f .

D Proof of Necessity of OT

Let $f((m_0, m_1), \mathbf{b}, \perp, \dots, \perp) = (\perp, m_{\mathbf{b}}, \perp, \dots, \perp)$, where HP has a pair of messages (m_0, m_1) as input, P_1 has a single input bit $\mathbf{b}$ and the rest of the participants have no input. The function is solitary i.e. only one party, namely P_1 obtains the output $m_{\mathbf{b}}$. Towards a contradiction, let us assume that π_f securely computes f in the HP-aided setting over an asynchronous network. Recall that this means the adversary could either corrupt at most $n - 1$ parties among $\{P_1, \dots, P_n\}$ maliciously or HP in a semi-honest manner. Consider the following scenario in an execution of π_f :

Scenario A: Suppose the adversary corrupts HP in a semi-honest manner. Further, the adversarial message scheduling is such that all incoming and outgoing messages from $\{P_2, \dots, P_n\}$ are delayed.

Claim. Suppose Scenario A occurs. Then the output of P_1 is $m_{\mathbf{b}}$ with overwhelming probability.

Proof. This follows from the correctness of π_f and the fact that we are in the semi-honest corruption case in Scenario A where the protocol must result in the output (i.e. abort is not allowed). Since the helper received messages from P_1 at every step and must have not been able to include inputs of the $(n - 1)$ parties $\{P_2, \dots, P_n\}$ (as it never received any message from them), we can conclude that this output must be computed on P_1 's input $\mathbf{b}$. Thus, the output in this scenario must be $m_{\mathbf{b}}$ with overwhelming probability.

Next, we describe a synchronous 2-party OT protocol π_{OT} between a sender S (with inputs (m'_0, m'_1)) and receiver R (with choice bit c) as follows. S does the following:

- Emulate the role of HP in π_f and compute messages accordingly using input $(m_0 = m'_0, m_1 = m'_1)$.
- Whenever π_f instructs to send a message to P_1 , send it to R .
- Whenever π_f instructs to send a message to $P_2, \dots, P_n$, send nothing.

R does the following:

- Emulate the role of P_1 in π_f and compute messages accordingly using input $b = c$.
- Whenever π_f instructs to send a message to HP , send it to S .
- Whenever π_f instructs to send a message to $P_2, \dots, P_n$, send nothing.

Claim. Consider an honest execution of π_{OT} over a synchronous network. The receiver R is able to compute the correct output m'_c with overwhelming probability.

Proof. We note that R 's view at the end of an honest execution of π_{OT} is identically distributed to the view of honest P_1 in Scenario A . This is because in Scenario A , the helper and P_1 received messages only from each other; they never received any message from any of the other parties. It follows from Claim [D](#) that this suffices to compute m'_c with overwhelming probability.

Claim. π_{OT} is an OT protocol over a synchronous network with semi-honest security against the sender and malicious security against the receiver.

Proof. Correctness of π_{OT} follows from Claim [D](#). Now suppose S is semi-honest. Security follows directly from security of π_f for the case when HP is semi-honest. Consider Scenario A in π_f . It follows from security of π_f that there exists a simulator $\mathcal{S}_{HP}$ that outputs a view indistinguishable to the view of semi-honest HP in Scenario A . Since the view of semi-honest HP in Scenario A is identically distributed to a semi-honest S in π_{OT} , we can therefore design a simulator $\mathcal{S}_S^{OT}$ that runs $\mathcal{S}_{HP}$ to create a view indistinguishable to the view of a semi-honest S in π_{OT} .

Next, we consider a maliciously corrupt R and note that security follows directly from security of π_f for the case (say Scenario B) where the adversary maliciously corrupts P_1 in the following way: The adversarial scheduling is such that both the incoming and outgoing messages of $\{P_2, \dots, P_n\}$ are arbitrarily delayed. Towards HP , P_1 behaves identical to maliciously corrupt R . Since π_f is secure against such a potentially malicious P_1 , there exists a simulator $\mathcal{S}_{P_1}$ that creates a view indistinguishable to the view of P_1 in Scenario B . Since the view of a corrupt R in π_{OT} is identically distributed to the view of corrupt P_1 in Scenario B , we can therefore design a simulator $\mathcal{S}_R^{OT}$ that runs $\mathcal{S}_{P_1}$ to create a view that is indistinguishable to the view of corrupt R in π_{OT} .

E Multi Sender Single Receiver OLE

In this section, we present the detailed ideal functionality for multi sender single receiver OLE.

Functionality $\mathcal{F}_{MSSR-OLE}$

- **Initialize:** On input $(\text{Init}, S_i, \mathbb{F})$ from S_i for all $i \in [1, n]$ and $(\text{Init}, R, \mathbb{F})$ from R , store $\mathbb{F}$.
- **Input:** On input (Input, a, c) from S_i , for all $i \in [1, n]$ and (Input, b) from R , store (a, b, c) .
- **Output:** On input (Output) from all the parties, send **Output** to the adversary. Wait for an input from the adversary; if this is 'deliver' then send $(a \cdot b - c)$ to R , and send $\perp$ to S_i for all $i \in [1, n]$. Else, if it received 'abort' from the adversary then it outputs $\perp$ to all the parties including the adversary.

Fig. 18: Ideal functionality for Multiple Sender Single Receiver OLE

Next, we present the detailed construction of multi sender single receiver OLE with security against a malicious sender.

Protocol $\Pi_{\text{MSSR-OLE}}$

Input: For all $i \in [1, n]$, S_i has common input a, c and R has input b .

Output: R receives b where $a \cdot b = c$ and for all $i \in [1, n]$, S_i does not have an output.

Primitive: Malicious sender OLE (Π_{OLE}), common key setup.

Protocol: Let Π_{OLE} be a malicious sender secure OLE protocol where S_{OLE} has input a, c and R_{OLE} has input b . Finally, R_{OLE} receives the output $a \cdot b = c$.

If $P = R$, then it does the following:

- P starts the protocol Π_{OLE} on behalf of R_{OLE} . If R_{OLE} sends a message msg to S , it sends msg to S_i for all $i \in [1, n]$.
- In Π_{OLE} , when R_{OLE} expects a message msg' from S_{OLE} , then P waits for a message msg'' from one sender. If it receives msg'' from a sender S_i , it discards all subsequent messages.
- P follows the same steps of R_{OLE} and output the same as R_{OLE} .
- Finally, Upon receiving an output d , it sends terminate to S_i for all $i \in [1, n]$.

If $P = S_i$ for all $i \in [1, n]$, then it does the following:

- P starts the protocol Π_{OLE} on behalf of S_{OLE} . If S_{OLE} sends a message msg to R_{OLE} , it sends msg to R .
- In Π_{OLE} , when S_{OLE} expects a message msg' from R_{OLE} , then P waits for a message msg'' from R .
- P follows the same steps of S_{OLE} .
- Finally, it terminates if it receives terminate from R .

Fig. 19: Malicious sender secure Multi Sender Single Receiver OLE

F Security Proofs

Functionality $\mathcal{F}_{\text{MPC}}$

The functionality is parametrized by a bit b such that $b = 1$ when the computational parties are malicious, and $b = 0$ when HP is semi-honest.

The functionality is also parametrized with the set $C \subset \mathcal{P}$ of corrupt parties when $b = 1$.

- Each computational party P_i for all $i \in [1, n]$ sends its input x_i to the functionality.
- The functionality receives sets H from the adversary such that $H \subset \mathcal{P}$ and $|H| \leq t$.
- The functionality sets the input of each $P_i \in H$ to the default value, that is, $x_i = 0$.
- If $b = 1$, then the functionality receives from the adversary a message $m \in \{\text{abort}, \text{continue}\}$.
- If $b = 1$ and $m = \text{abort}$, then the functionality sends abort to all parties and HP. Otherwise, it computes $(y_0, y_1, \dots, y_n) = f(x_1, \dots, x_n)$ and sends y_i to P_i for each $P_i \in \mathcal{P}$. It also sends y_0 to HP.

Fig. 20: MPC Functionality

F.1 Security of Pollux

Let us assume that the adversary corrupts parties $\{P_1, \dots, P_t\}$. Then the set $\mathcal{H} = \mathcal{P} \setminus \{P_1, \dots, P_t\}$ denotes the honest parties.

Simulator $\mathcal{S}_{\mathcal{A}}^{\text{Pollux}}$

Malicious Let $\mathcal{A}$ be a malicious adversary corrupting up to $t = (n - 1)/2$ parties among the computing parties $\mathcal{P} = \{P_1, \dots, P_n\}$. Therefore there are at least $t + 1$ honest parties, let $\mathcal{H}$ be the set of honest parties in $\mathcal{P}$. In this case, recall that HP is also honest. Let $\mathcal{S}_{\mathcal{A}}$ denote the simulator for this case.

- **Preprocessing:** $\mathcal{S}_{\mathcal{A}}^{\text{Pollux}}$ simulates the preprocessing phase of the protocol by acting on behalf of HP as per the protocol specifications.
- **Input:** Let P_d be the input provider.

- **Case I:** $P_d \notin \mathcal{H}$. In the online phase, $\mathcal{S}_A^{\text{Pollux}}$ receives $\mathbf{q}_v$ from P_d (since $P_d \notin \mathcal{H}$, thus corrupted by $\mathcal{A}$) and sends $\mathbf{q}_v$ to all the parties. $\mathcal{S}_A^{\text{Pollux}}$ extracts P_d 's input $\mathbf{v} = \mathbf{q}_v - \mathbf{r}_v - \delta_v$. Note that $\mathbf{r}_v$ is the common random pad known to the parties in $\mathcal{H}$ and δ_v is known to HP from the preprocessing phase.
- **Case II:** $P_d \in \mathcal{H}$. In the online phase, $\mathcal{S}_A^{\text{Pollux}}$ samples a random value $\mathbf{q}_v$ and sends it to all the parties on behalf of HP.
- **Multiplication:** In the online phase, $\mathcal{S}_A^{\text{Pollux}}$ receives shares of $[\mathbf{q}_z]_i$ from the corrupt P_i . If $[\mathbf{q}_z]_i \neq \mathbf{m}_x \mathbf{m}_y - \mathbf{m}_x [\delta_y]_i - \mathbf{m}_y [\delta_x]_i + [\delta_{xy}]_i + [\delta_z]_i + \mathbf{r}_z$ then it sets $\mathcal{F}_{\text{MPC}}$ message as **abort**. $\mathcal{S}_A$ sends $\mathbf{m}_z$ to all the parties on behalf of $\mathcal{S}_A^{\text{Pollux}}$ by following the protocol steps honestly.
- **Verification:** $\mathcal{S}_A^{\text{Pollux}}$ simulates $\mathcal{F}_{\text{Rand}}$ and obtains $\boldsymbol{\rho}$. Further, it obtains a sharing of zero, i.e., P_i 's share is α_i where $\alpha_i = f(i)$ for some t degree polynomial such that $f(0) = 0$. For all parties P_i corrupted by $\mathcal{A}$, $\mathcal{S}_A^{\text{Pollux}}$ receives $[\omega_z]_i$. $\mathcal{S}_A^{\text{Pollux}}$ checks if $[\omega_z]_i = \rho_0 \cdot \alpha_i + \sum_{j \in [1, m]} \rho_j \cdot ([\mathbf{t}_{\mathbf{m}_{z_j}}]_i - \mathbf{m}_{z_j}^* \cdot [\Delta]_i)$, where $\mathbf{m}_{z_j}^*$ be the unique opened value. Note that $[\mathbf{t}_{\mathbf{m}_{z_j}}]_i = -\mathbf{m}_{x_j} \cdot [\mathbf{t}_{y_j}]_i - \mathbf{m}_{y_j} \cdot [\mathbf{t}_{x_j}]_i + [\mathbf{t}_{x_j y_j}]_i + [\mathbf{t}_{z_j}]_i + [\Delta]_i \cdot (\mathbf{m}_{x_j} \cdot \mathbf{m}_{y_j})$, where $\mathbf{m}_{x_j}, \mathbf{m}_{y_j}$ for all $j \in [1, \ell]$ are held by the parties in $\mathcal{H}$ and $[\Delta], [\mathbf{t}_{x_j}], [\mathbf{t}_{y_j}], [\mathbf{t}_{x_j y_j}], [\mathbf{t}_{z_j}]$ are held by HP.
- **Output:** If the above verification fails, $\mathcal{S}_A^{\text{Pollux}}$ sets the adversary's message to $\mathcal{F}_{\text{MPC}}$ as **abort**. $\mathcal{S}_A^{\text{Pollux}}$ invokes $\mathcal{F}_{\text{MPC}}$ with the adversary's input ($\perp$ or $\mathbf{v}$). Recall that $\mathcal{S}_A^{\text{Pollux}}$ had extracted corrupt P_i 's input during the input stage. If the adversary's input is not **abort**, then $\mathcal{F}_{\text{MPC}}$ gives $\mathbf{y}$ as the output, then it computes $\delta_y = \mathbf{m}_y - \mathbf{y}$ and sends on behalf of HP.

Fig. 21: Simulator $\mathcal{S}_A^{\text{Pollux}}$ for Pollux for a malicious adversary corrupting up to $t(\leq (n-1)/2)$ parties among n parties

Indistinguishability.

Fig. 21 describes the simulation steps when a malicious adversary corrupts up to $t(\leq (n-1)/2)$ parties among n parties. In this scenario HP and parties in $\mathcal{H}$ are honest. Consider $\mathcal{A}$ be a malicious adversary corrupting up to t parties (i.e. parties in $\mathcal{P} \setminus \mathcal{H}$) among the party set $\mathcal{P}$. For input gates, if P_d is corrupt, then $\mathcal{S}_A^{\text{Pollux}}$ does not send any messages. Thus, the view is indistinguishable trivially. If P_d is honest then $\mathcal{S}_A^{\text{Pollux}}$ sends a random value $\tilde{\mathbf{q}}_v$ to $\mathcal{A}$. In the real protocol, P_d sends $\mathbf{q}_v$ to HP, which is forwarded to all the parties. Here $\mathbf{q}_v = \mathbf{m}_v + \mathbf{r}_v$, where $\mathbf{m}_v = \mathbf{v} + \delta_v$. Since δ_v is sampled uniformly, therefore $\mathbf{q}_v$ is distributed uniformly. Hence the distribution of $\mathbf{q}_v$ and $\tilde{\mathbf{q}}_v$ are identical.

For a multiplication gate, $\mathcal{S}_A^{\text{Pollux}}$ receives shares of $\mathbf{q}_z$ from all the corrupt parties. If it receives an incorrect share, then $\mathcal{S}_A^{\text{Pollux}}$ sets the $\mathcal{F}_{\text{MPC}}$ message as **abort**, and in the real protocol, the verification fails with a very high probability. $\mathcal{S}_A^{\text{Pollux}}$ sends the reconstructed $\mathbf{q}_z$ to all the parties by honestly following the protocol steps.

Next, we analyze the difference between the verification check carried out by the simulator $\mathcal{S}_A^{\text{Pollux}}$ and the verification check in the real-world protocol. In the verification step, $\mathcal{A}$ can introduce an error in $\mathbf{m}_{z_j}$. However, it can obtain a sharing of the corresponding tag with negligible probability $(\frac{1}{|\mathbb{F}|})$. Thus, consider

a $j \in [1, m]$ such that $[\mathbf{t}_{\mathbf{m}_{z_j}}]_i - \mathbf{m}_{z_j}^* \cdot [\Delta]_i \neq 0$, however

$$\sum_{k=1}^{t+1} \lambda_{i_k} \cdot \left[\rho_0 \cdot \alpha_{i_k} + \sum_{j \in [m]} \rho_j \cdot ([\mathbf{t}_{\mathbf{m}_{z_j}}]_{i_k} - \mathbf{m}_{z_j}^* \cdot [\Delta]_{i_k}) \right] = 0 \quad (1)$$

Where $P_{i_1}, \dots, P_{i_{t+1}}$ are the parties first $t+1$ parties whose messages were delivered to HP and λ_{i_k} are the corresponding Lagrange's coefficients. Equation (1) can be satisfied for some choices of $\boldsymbol{\rho}$. To elaborate, consider $\xi_0 = \sum_{k=1}^{t+1} \lambda_{i_k} \cdot \alpha_{i_k}$ and $\xi_j = \sum_{k=1}^{t+1} \lambda_{i_k} \cdot ([\mathbf{t}_{\mathbf{m}_{z_j}}]_{i_k} - \mathbf{m}_{z_j}^* \cdot [\Delta]_{i_k})$ for all $j \in [1, m]$, set $\boldsymbol{\xi} = (\xi_0, \xi_1, \dots, \xi_m)$. Then $T_{\boldsymbol{\xi}}(\boldsymbol{\rho}) = \sum_{j=0}^m \xi_j \cdot \rho_j$. $T_{\boldsymbol{\xi}}$ is a non-zero linear map, then $\dim(\ker(T_{\boldsymbol{\xi}})) \leq m$. Therefore, the probability that a randomly sampled $\boldsymbol{\rho}$ belongs to $\ker(T_{\boldsymbol{\xi}}) \leq \frac{|\mathbb{F}|^m}{|\mathbb{F}|^{m+1}} = \frac{1}{|\mathbb{F}|}$. $\mathcal{S}_A^{\text{Pollux}}$ gets $[\omega_z]_i$ from P_i corrupted by $\mathcal{A}$. In the real protocol, $[\omega_z]_i \neq \rho_0 \cdot \alpha_i + \sum_{j \in [m]} \rho_j \cdot ([\mathbf{t}_{\mathbf{m}_{z_j}}]_i - \mathbf{m}_{z_j}^* \cdot [\Delta]_i)$. $\mathcal{A}$ can introduce an error ϵ_i in the $[\omega_z]_i$ term such that $\sum_{k=1}^{t+1} \lambda_{i_k} \cdot ([\omega_z]_i + \epsilon_i) = 0$. In that case, in the real protocol, $\mathcal{A}$ cheats successfully. In the simulated world, the simulator aborts. However, $\mathcal{A}$ can find such ϵ_i with probability $\frac{1}{|\mathbb{F}|}$. Therefore, $\mathcal{A}$ can successfully

cheat with probability at most $\frac{2}{|\mathbb{F}|}$ which is negligible. Thus the real world view is indistinguishable from the simulated view.

Simulator $\mathcal{S}_{\text{HP}}^{\text{Pollux}}$

Semi-Honest Let HP be semi-honest. Thus all the computational parties are honest. Let $\mathcal{S}_{\text{HP}}$ be the simulator. Since HP does not have any input, $\mathcal{S}_{\text{HP}}$ works as follows on input y where y is the output of the functionality.

- **Preprocessing:** $\mathcal{S}_{\text{HP}}^{\text{Pollux}}$ acts on behalf of the parties in $\mathcal{P}$ and receives the values from HP.
- **Input:** $\mathcal{S}_{\text{HP}}^{\text{Pollux}}$ samples a random value $\tilde{q}_v$ for an input wire on behalf of an input provider. Then it receives the same $\tilde{q}_v$ from HP.
- **Multiplication:** For multiplication gates, $\mathcal{S}_{\text{HP}}^{\text{Pollux}}$ sends a Shamir sharing of a random value q_z to HP on behalf of all the parties. HP reconstructs using the first $t + 1$ messages and sends the reconstructed value to all the parties.
- **Verification:** $\mathcal{S}_{\text{HP}}^{\text{Pollux}}$ samples a random sharing of zero, $[\tilde{\omega}]_i$ and sends it to HP.

Fig. 22: Simulator $\mathcal{S}_{\text{HP}}^{\text{Pollux}}$ for Pollux for a semi-honest HP

Fig. 22 describes the simulation steps when the HP is semi-honest. In this case, all parties in $\mathcal{P}$ are honest. Since HP does not have any input to the protocol, the simulated view of the preprocessing step of the protocol is identical to the real view of the preprocessing phase. For an input gate, $\mathcal{S}_{\text{HP}}^{\text{Pollux}}$ samples and sends a randomly sampled $\tilde{q}_v$. In the real protocol, HP receives $q_v = m_v + r_v$, where r_v is uniformly sampled. Therefore, q_v is also uniformly distributed. Thus, the distribution of q_v and $\tilde{q}_v$ are identical. For a multiplication gate, $\mathcal{S}_{\text{HP}}^{\text{Pollux}}$ samples and sends $[\tilde{q}_z]_i$ to HP on behalf of P_i . In the real protocol, P_i sends $[q_z]_i$ to HP, where $[q_z]_i = [m_z]_i + r_v$ where r_v is a randomly sampled value. Therefore, $[q_z]_i$ is also uniformly distributed. Hence, for all $i \in [1, n]$, $[\tilde{q}_z]_i$ and $[q_z]_i$ are identically distributed. In the verification step, $\mathcal{S}_{\text{HP}}^{\text{Pollux}}$ samples a random Shamir sharing of zero $[\tilde{\omega}]_i$. In the real protocol, P_i sends $[\omega]_i$ such that $\sum_{k=1}^{t+1} \lambda_{i_k} \cdot [\omega]_{i_k} = 0$, where $P_{i_1}, \dots, P_{i_{t+1}}$ are the first $t + 1$ whose messages were delivered to HP and λ_{i_k} are the corresponding Lagrange's coefficients, for $k \in [1, t + 1]$. Since all the parties follow the protocol. Also, $[\omega]_i = \rho_0 \cdot \alpha_i + \sum_{j=1}^m \rho_j [\omega_{z_j}]_i$, where $\{\alpha_i\}_{i \in [1, n]}$ is a random Shamir sharing of zero. Therefore $[\omega]_i$ is uniformly distributed such that $\sum_{k=1}^{t+1} \lambda_{i_k} \cdot [\omega]_{i_k} = 0$. Therefore, $[\omega]_i$ and $[\tilde{\omega}]_i$ are identically distributed.

F.2 Security of Castor

In this section, we provide the security proof of our construction Castor. We begin by considering a malicious adversary $\mathcal{A}$ that corrupts all but one party. The protocol Castor is divided into two phases: preprocessing (Π_{Preproc}) and online (Π_{Online}). The preprocessing phase Π_{Preproc} makes use of the sub-protocol $\Pi_{\text{mult-ARSS}}$, which in turn relies on $\Pi_{\text{mult-RSS}}$.

We first prove the security of $\Pi_{\text{mult-RSS}}$ in the $\mathcal{F}_{\text{MSSR-OLE}}$ -hybrid model. Then, using the security of $\Pi_{\text{mult-RSS}}$, we establish the security of $\Pi_{\text{mult-ARSS}}$. Building upon this, we prove the security of Π_{Preproc} based on the security of $\Pi_{\text{mult-ARSS}}$. Finally, we provide a simulation-based proof for the online phase Π_{Online} . By combining these results using a hybrid argument, we establish the overall security of Castor.

Subsequently, we consider the case where the helper HP is semi-honest. We follow a similar proof structure to establish security in this setting.

Security Proof of Castor against a malicious adversary $\mathcal{A}$

Simulator $\mathcal{S}_{\mathcal{A}}^{\Pi_{\text{mult-RSS}}}$

- $\mathcal{S}_{\mathcal{A}}^{\Pi_{\text{mult-RSS}}}$ follows the protocol on behalf of HP and the honest party P_h . $\mathcal{S}_{\mathcal{A}}^{\Pi_{\text{mult-RSS}}}$ holds $[x]_i, [y]_i$ for all $i \in [0, n] \setminus \{h\}$. It uniformly samples $[x]_h, [y]_h$.
- It simulates $\mathcal{F}_{\text{MSSR-OLE}}$ where it acts on behalf of receiver (HP) with the inputs $\sum_{i=1}^n [x]_i$. Let u be the output of $\mathcal{F}_{\text{MSSR-OLE}}$.

- It simulates $\mathcal{F}_{\text{MSSR-OLE}}$ where it acts on behalf of receiver (HP) with the inputs $\sum_{i=1}^n [y]_i$. Let w be the output of $\mathcal{F}_{\text{MSSR-OLE}}$.
- It sends **abort** to $\mathcal{F}_{\text{MPC}}$ if any of the above simulation results in abort.
- For all $i \in [1, n-1] \setminus \{h\}$, $\mathcal{S}_{\mathcal{A}}^{\Pi_{\text{mult-RSS}}}$ samples $[z^{(0i)}]_i$, $[z^{(i0)}]_i$ and $[\eta]_i$ with P_i using the common key.
- It invokes $\mathcal{F}_{\text{MPC}}$ functionality with $[x]_i, [y]_i$ for all $i \in [0, n] \setminus \{h\}$ inputs and gets either $\perp$ or $[z]_i$ for all $i \in [0, n] \setminus \{h\}$ as output.
- It samples uniform $[z^{(0i)}]_n$, $[z^{(i0)}]_n$ and $[\eta]_n$ such that $z_n = [x]_n [y]_n + [z^{(0i)}]_n + [z^{(i0)}]_n + [\eta]_n$.
- It sends $[z^{(0i)}]_n + [z^{(i0)}]_n + [\eta]_n$ to P_n .

Fig. 23: Simulator $\mathcal{S}_{\mathcal{A}}^{\Pi_{\text{mult-RSS}}}$ for $\Pi_{\text{mult-RSS}}$ for a malicious adversary corrupting majority of $\mathcal{P}$

Indistinguishability In the protocol $\Pi_{\text{mult-RSS}}$, the view of the adversary $\mathcal{A}$ is indistinguishable from the view generated by the simulator $\mathcal{S}_{\mathcal{A}}^{\Pi_{\text{mult-RSS}}}$ in the $\mathcal{F}_{\text{MSSR-OLE}}$ -hybrid model. Specifically, $\mathcal{S}_{\mathcal{A}}^{\Pi_{\text{mult-RSS}}}$ emulates the honest party's behavior by executing $\mathcal{F}_{\text{MSSR-OLE}}$ on behalf of the receiver. It then communicates the value $[z^{(0i)}]_n + [z^{(i0)}]_n + [\eta]_n$ to P_n .

In the real execution, the honest party HP interacts with $\mathcal{F}_{\text{MSSR-OLE}}$ as the receiver and subsequently sends the same value $[z^{(0i)}]_n + [z^{(i0)}]_n + [\eta]_n$ to P_n . Therefore, all messages received by $\mathcal{A}$ are either obtained directly from $\mathcal{F}_{\text{MSSR-OLE}}$ or are sampled from identical distributions in both the real and simulated executions. Consequently, the view of $\mathcal{A}$ in the real and ideal worlds are indistinguishable, and hence $\Pi_{\text{mult-RSS}}$ is secure in the $\mathcal{F}_{\text{MSSR-OLE}}$ -hybrid model.

Simulator $\mathcal{S}_{\mathcal{A}}^{\Pi_{\text{mult-ARSS}}}$

- $\mathcal{S}_{\mathcal{A}}^{\Pi_{\text{mult-ARSS}}}$ follows the protocol on behalf of HP and the honest party P_h . $\mathcal{S}_{\mathcal{A}}^{\Pi_{\text{mult-ARSS}}}$ holds $[x]_i, [y]_i, [t_x]_i, [t_y]_i, [\Delta]_i$ for all $i \in [0, n] \setminus \{h\}$. It uniformly samples $[x]_h, [y]_h, [t_x]_h, [t_y]_h, [\Delta]_h$.
- It simulates $\mathcal{F}_{\text{mult-RSS}}$ where it acts on behalf of honest HP and P_h with inputs $[x], [y]$ and obtains output $[xy]$.
- It simulates $\mathcal{F}_{\text{mult-RSS}}$ where it acts on behalf of honest HP and P_h with inputs $[xy], [\Delta]$ and obtains output $[t_{xy}]$.
- It generates $\langle \hat{y} \rangle$ by following the steps, it generates $[\hat{y}]$, followed by simulating $\mathcal{F}_{\text{mult-RSS}}$ where it acts on behalf of honest HP and P_h with inputs $[\hat{y}], [\Delta]$ and obtains output $[t_{\hat{y}}]$.
- It simulates $\mathcal{F}_{\text{mult-RSS}}$ where it acts on behalf of honest HP and P_h with inputs $[x], [\hat{y}]$ and obtains output $[x\hat{y}]$.
- It simulates $\mathcal{F}_{\text{mult-RSS}}$ where it acts on behalf of honest HP and P_h with inputs $[x\hat{y}], [\Delta]$ and obtains output $[t_{x\hat{y}}]$.
- $\mathcal{S}_{\mathcal{A}}^{\Pi_{\text{mult-ARSS}}}$ receives $[\rho]_0$ from each P_i and upon receiving it from the first party, it computes ρ by following the protocol steps and sends it to all.
- It then computes $\langle \tau \rangle$ as per the protocol on behalf of HP and P_h .
- $\mathcal{S}_{\mathcal{A}}^{\Pi_{\text{mult-ARSS}}}$ receives $[\tau]_0$ from each P_i and upon receiving it from the first party, it computes τ by following the protocol steps and sends it to all.
- (Simulating one-shot verification) Simulator follows the protocol steps as defined for ℓ multiplications in a batch. Specifically, the simulator receives $\langle \hat{\rho} \rangle_0, \langle \hat{\tau} \rangle_0$ from the party that sends it first and reconstructs $\hat{\rho}, \hat{\tau}, t_{\hat{\rho}}, t_{\hat{\tau}}$ and sends it to all.
- It computes $[u]$ and $[v]$ as per the protocol steps on behalf of HP, P_h . Then it reconstructs u, v similar by receiving $[u]_0, [v]_0$ from the party that sends it first and sends it to all.
- Corrupt parties output what the adversary outputs.

Fig. 24: Simulator $\mathcal{S}_{\mathcal{A}}$ for $\Pi_{\text{mult-ARSS}}$ for a malicious adversary corrupting majority of $\mathcal{P}$

Indistinguishability In the protocol $\Pi_{\text{mult-ARSS}}$, the view of the adversary $\mathcal{A}$ is indistinguishable from the view generated by the simulator $\mathcal{S}_{\mathcal{A}}^{\Pi_{\text{mult-ARSS}}}$ in the $\mathcal{F}_{\text{mult-RSS}}$ -hybrid model. Specifically, $\mathcal{S}_{\mathcal{A}}^{\Pi_{\text{mult-ARSS}}}$ emulates the

behavior of the honest parties by invoking $\mathcal{F}_{\text{mult-RSS}}$ on their behalf. In the subsequent steps, the simulator proceeds by honestly following the remaining steps of the protocol.

Since all messages received by $\mathcal{A}$ are either outputs from $\mathcal{F}_{\text{mult-RSS}}$ or derived from computations that are identically distributed in both the real and ideal worlds, the view of $\mathcal{A}$ in the simulated execution is indistinguishable from its view in the real execution. Therefore, $\Pi_{\text{mult-ARSS}}$ is secure in the $\mathcal{F}_{\text{mult-RSS}}$ -hybrid model.

Simulator for $\Pi_{\text{Preproc}}^{\text{Castor}}$ We now give the simulator for the preprocessing phase, $\Pi_{\text{Preproc}}^{\text{Castor}}$, of the dishonest majority protocol **Castor**. This is followed by the indistinguishability argument to complete the proof of its security.

Simulator $\mathcal{S}_{\mathcal{A}}^{\Pi_{\text{Preproc}}^{\text{Castor}}}$

- $\mathcal{S}_{\mathcal{A}}^{\Pi_{\text{Preproc}}^{\text{Castor}}}$ follows the protocol on behalf of HP and the honest party P_h .
- For an input gate, consider P_d be the input provider and v be the input. It simulates in the following way:
 - If P_h be the input provider, then it samples random shares of the mask using appropriate common keys such that $\mathcal{S}_{\mathcal{A}}^{\Pi_{\text{Preproc}}^{\text{Castor}}}$ holds the random mask in clear on behalf of P_h . It generates replicated sharing of the mask $[\delta_v]$.
 - If P_h is not the input provider, then it samples random shares of the mask using appropriate common keys. It generates replicated sharing of the mask $[\delta_v]$.
 - It simulates $\mathcal{F}_{\text{mult-RSS}}$ with inputs $([\Delta], [\delta_v])$.
- For an addition gate, it simulates by following the protocol steps.
- For an multiplication gate, consider x and y be the inputs and δ_x, δ_y be the corresponding masks. It simulates $\mathcal{F}_{\text{mult-ARSS}}$ with inputs $[\delta_x], [\delta_y]$.
- For an output, it generates $[\cdot]$ -sharing of a random mask δ_v using appropriate common keys.

Fig. 25: Simulator $\mathcal{S}_{\mathcal{A}}$ for $\Pi_{\text{Preproc}}^{\text{Castor}}$ for a malicious adversary corrupting majority of $\mathcal{P}$

Indistinguishability The indistinguishability of the preprocessing phase $\Pi_{\text{Preproc}}^{\text{Castor}}$ directly follows from the security of the sub-protocols $\Pi_{\text{mult-RSS}}$ and $\Pi_{\text{mult-ARSS}}$, which are simulated in the $\mathcal{F}_{\text{mult-RSS}}$ - and $\mathcal{F}_{\text{mult-ARSS}}$ -hybrid models, respectively. The simulator $\mathcal{S}_{\mathcal{A}}^{\Pi_{\text{Preproc}}^{\text{Castor}}}$ invokes these ideal functionalities and emulates the honest parties' behavior accordingly. All remaining steps of the protocol are executed honestly by the simulator and are identical to those in the real execution.

Therefore, the view generated by $\mathcal{S}_{\mathcal{A}}^{\Pi_{\text{Preproc}}^{\text{Castor}}}$ is indistinguishable from the view of $\mathcal{A}$ in the real execution of $\Pi_{\text{Preproc}}^{\text{Castor}}$. This establishes the security of the preprocessing phase in the hybrid model.

Simulator for the online phase

Simulator $\mathcal{S}_{\mathcal{A}}^{\Pi_{\text{Online}}^{\text{Castor}}}$

- *Preprocessing:* $\mathcal{S}_{\mathcal{A}}^{\Pi_{\text{Online}}^{\text{Castor}}}$ simulates the preprocessing phase by executing $\mathcal{S}_{\mathcal{A}}^{\Pi_{\text{Preproc}}^{\text{Castor}}}$ and prepare for the online phase.
- *Input gates:*
 - Case I: Let P_h be the input provider. Then $\mathcal{S}_{\mathcal{A}}^{\Pi_{\text{Online}}^{\text{Castor}}}$ samples a random value m_v on behalf of P_h and sends it to all the parties on behalf of HP.
 - Case II: Let P_i , for $i \neq h$, be the input provider. Then $\mathcal{S}_{\mathcal{A}}^{\Pi_{\text{Online}}^{\text{Castor}}}$ receives m_v on behalf of HP from $\mathcal{A}$ and sends it to all the parties on behalf of HP. Furthermore, it extracts the input, as it has all the shares of δ_v (P_i has all the shares, which are common with P_h and HP together combined).
- *Addition gates:* Since the addition gates are non-interactive, nothing to simulate.

- *Multiplication gates:* Let x, y are the input values of the multiplication gate and z be the output. Parties hold $\llbracket x \rrbracket, \llbracket y \rrbracket$. $\mathcal{S}_A^{\Pi_{\text{Online}}^{\text{Castor}}}$ computes $\llbracket m_z \rrbracket_0$ by following the protocol on behalf of P_h and sends it to HP. $\mathcal{S}_A^{\Pi_{\text{Online}}^{\text{Castor}}}$ samples a random m_z and sends it to all the parties on behalf of HP.
- *Verification:* $\mathcal{S}_A^{\Pi_{\text{Online}}^{\text{Castor}}}$ simulates $\mathcal{F}_{\text{Rand}}$ and obtains ρ . $\mathcal{S}_A^{\Pi_{\text{Online}}^{\text{Castor}}}$ it sends $\llbracket \omega_z \rrbracket_0$ to HP on behalf of P_h . It invokes the functionality $\mathcal{F}_{\text{MPC}}$, if it outputs $\perp$, then it outputs 0, else if the functionality outputs y , then it outputs 1.
- *Output:* If the output verification step is 1, it sends the received value y from the functionality to all the parties. Else it sends $\perp$ to all the parties.

Fig. 26: Simulator $\mathcal{S}_A^{\Pi_{\text{Online}}^{\text{Castor}}}$ for $\Pi_{\text{Online}}^{\text{Castor}}$ for a malicious adversary corrupting majority of $\mathcal{P}$

Indistinguishability Given the preprocessing data generated by $\Pi_{\text{Preproc}}^{\text{Castor}}$, the online phase $\Pi_{\text{Online}}^{\text{Castor}}$ securely realizes the ideal functionality $\mathcal{F}_{\text{MPC}}$. The simulator $\mathcal{S}_A^{\Pi_{\text{Online}}^{\text{Castor}}}$ proceeds as follows:

For input sharing, if the input provider is honest, $\mathcal{S}_A^{\Pi_{\text{Online}}^{\text{Castor}}}$ generates a uniformly random masked input m_v and sends it on behalf of the honest party via the trusted party HP. In the real execution, since the mask δ_v is sampled uniformly at random, the masked input $m_v = v + \delta_v$ is also uniformly distributed and independent of the actual input. Hence, the adversary's view is indistinguishable in both the real and simulated executions.

If the input provider is corrupted, $\mathcal{S}_A^{\Pi_{\text{Online}}^{\text{Castor}}}$ receives the masked input directly from the adversary, extracts the actual input, and forwards the received message to all parties. This step is identical to the real protocol, ensuring that the adversary's view remains unchanged.

For multiplication gates, $\mathcal{S}_A^{\Pi_{\text{Online}}^{\text{Castor}}}$ first computes $\llbracket m_z \rrbracket_0$ following the honest party's steps in the protocol and receives the corresponding message from $\mathcal{A}$. Upon receiving a single message from $\mathcal{A}$, the simulator samples a fresh random value m_z and sends it to all parties. If the adversary's message does not match the expected $\llbracket m_z \rrbracket_0$, the simulator reports $\perp$ as $\mathcal{A}$'s input to $\mathcal{F}_{\text{MPC}}$. In the real execution, the mask δ_z used for computing m_z is also chosen uniformly at random, ensuring that m_z is distributed identically in both worlds.

For verification, the simulator emulates HP honestly by executing the verification steps as specified in the protocol. If any multiplication gate resulted in a mismatch (i.e., $\mathcal{A}$'s message is $\perp$), the simulator invokes $\mathcal{F}_{\text{MPC}}$ with input $\perp$ and sends $\perp$ to all parties. Otherwise, it submits the malicious party's input to $\mathcal{F}_{\text{MPC}}$, receives the output y , and delivers this output to all parties.

Thus, the view of $\mathcal{A}$ in the simulated execution is indistinguishable from the view in the real execution, establishing the security of $\Pi_{\text{Online}}^{\text{Castor}}$ in realizing $\mathcal{F}_{\text{MPC}}$.

Security Proof of Castor against a semi-honest HP

Simulator $\mathcal{S}_{\text{HP}}^{\Pi_{\text{mult-RSS}}}$

- $\mathcal{S}_{\text{HP}}^{\Pi_{\text{mult-RSS}}}$ follows the protocol on behalf of the parties in $\mathcal{P}$. $\mathcal{S}_{\text{HP}}^{\Pi_{\text{mult-RSS}}}$ has $\{\llbracket x \rrbracket_i, \llbracket y \rrbracket_i, \llbracket z \rrbracket_i\}_{i \neq 0}$ as inputs. It samples $\llbracket x \rrbracket_0, \llbracket y \rrbracket_0$.
- It samples $\llbracket z^{(i0)} \rrbracket_0, \llbracket z^{(0i)} \rrbracket_0$ on behalf of the parties in $\mathcal{P}$ using the common key setup.
- It simulates $\mathcal{F}_{\text{MSSR-OLE}}$ where it acts on behalf n senders $(P_1, \dots, P_n)$ with the inputs $\llbracket y \rrbracket_0$ and $\llbracket z^{(i0)} \rrbracket_0$. Let u be the output of $\mathcal{F}_{\text{MSSR-OLE}}$ that HP receives.
- It simulates $\mathcal{F}_{\text{MSSR-OLE}}$ where it acts on behalf n senders $(P_1, \dots, P_n)$ with the inputs $\llbracket x \rrbracket_0$ and $\llbracket z^{(0i)} \rrbracket_0$. Let w be the output of $\mathcal{F}_{\text{MSSR-OLE}}$ that HP receives.
- It samples $\llbracket z^{(i0)} \rrbracket_i, \llbracket z^{(0i)} \rrbracket_i$ and $\llbracket \eta \rrbracket_i$ for all $i \in [1, n-1]$, on behalf of P_i using the common key with HP and receives $\llbracket z^{(i0)} \rrbracket_n + \llbracket z^{(0i)} \rrbracket_n \llbracket \eta \rrbracket_n$ from HP on behalf of P_n .

Fig. 27: Simulator $\mathcal{S}_{\text{HP}}$ for $\Pi_{\text{mult-RSS}}$ for a semi-honest HP

Indistinguishability The simulator $\mathcal{S}_{\text{HP}}^{\Pi_{\text{mult-RSS}}}$ acts on behalf of the parties in $\mathcal{P}$ and interacts with the semi-honest trusted party HP to simulate the execution of $\Pi_{\text{mult-RSS}}$. The simulator is given the inputs and outputs of HP, namely $[x]_i, [y]_i, [z]_i \neq 0$. It completes the input by sampling $[x]_0, [y]_0$ uniformly at random.

To simulate the execution, $\mathcal{S}_{\text{HP}}^{\Pi_{\text{mult-RSS}}}$ emulates the behavior of the honest parties by invoking $\mathcal{F}_{\text{MSSR-OLE}}$ on behalf of the n senders. At the end of the execution, it receives the expected message from the semi-honest HP.

Since all messages in the real execution are either generated by $\mathcal{F}_{\text{MSSR-OLE}}$ or derived from inputs sampled from identical distributions, and since HP is semi-honest and follows the protocol honestly, the view generated by the simulator is indistinguishable from the view in the real execution. Therefore, $\Pi_{\text{mult-RSS}}$ is secure in the $\mathcal{F}_{\text{MSSR-OLE}}$ -hybrid model against a semi-honest HP.

Simulator $\mathcal{S}_{\text{HP}}^{\Pi_{\text{mult-ARSS}}}$

- $\mathcal{S}_{\text{HP}}^{\Pi_{\text{mult-ARSS}}}$ follows the protocol on behalf of the parties in $\mathcal{P}$. $\mathcal{S}_{\text{HP}}^{\Pi_{\text{mult-ARSS}}}$ has $\langle x \rangle_0, \langle y \rangle_0, \langle z \rangle_0$ as inputs.
- It samples $[x]_0, [y]_0, [t_x]_0, [t_y]_0$ on behalf of parties in $\mathcal{P}$.
- It follows the protocol steps by running $\mathcal{S}_{\text{HP}}^{\Pi_{\text{mult-RSS}}}$ corresponding to the instance $\Pi_{\text{mult-RSS}}([x], [y])$. Subsequently, it runs $\mathcal{S}_{\text{HP}}^{\Pi_{\text{mult-RSS}}}$ for the instance $\Pi_{\text{mult-RSS}}([xy], [\Delta])$ to generate $[t_{xy}]$.
- It runs the simulator for Π_{RandARSS} to generate $\langle \hat{y} \rangle$.
- It follows the protocol steps by running $\mathcal{S}_{\text{HP}}^{\Pi_{\text{mult-RSS}}}$ corresponding to the instance $\Pi_{\text{mult-RSS}}([x], [\hat{y}])$. Subsequently, it runs $\mathcal{S}_{\text{HP}}^{\Pi_{\text{mult-RSS}}}$ for the instance $\Pi_{\text{mult-RSS}}([x\hat{y}], [\Delta])$ to generate $[t_{x\hat{y}}]$.
- It computes $\langle \rho \rangle$ on behalf of the parties in $\mathcal{P}$ and sends $[\rho]_0$ to the adversary on behalf of each $P_i \in \mathcal{P}$. It then receives ρ from the adversary.
- Subsequently, it computes $\langle \tau \rangle$ on behalf of parties in $\mathcal{P}$ and sends $[\tau]_0$ to the adversary on behalf of each $P_i \in \mathcal{P}$. It receives τ from the adversary.
- (Simulating one-shot verification) Simulator computes $\langle \hat{\rho} \rangle$ and $\langle \hat{\tau} \rangle$ on behalf of the parties and sends $[\hat{\rho}]_0, [\hat{\tau}]_0$ and $[t_{\hat{\rho}}]_0, [t_{\hat{\tau}}]_0$ to the adversary.
- It receives from the adversary $\hat{\rho}, \hat{\tau}$ and the tags $t_{\hat{\rho}}, t_{\hat{\tau}}$.
- It computes $[u], [v]$ on behalf of the honest parties as per the protocol steps and sends $[u]_0, [v]_0$ to the adversary on behalf of each $P_i \in \mathcal{P}$.
- It receives from the adversary u, v .

Fig. 28: Simulator $\mathcal{S}_{\text{HP}}$ for $\Pi_{\text{mult-ARSS}}$ for a semi-honest HP

Indistinguishability In the protocol $\Pi_{\text{mult-ARSS}}$, the view of the adversary HP is indistinguishable from the view generated by the simulator $\mathcal{S}_{\text{HP}}^{\Pi_{\text{mult-ARSS}}}$ in the $\mathcal{F}_{\text{mult-RSS}}$ -hybrid model. Specifically, $\mathcal{S}_{\text{HP}}^{\Pi_{\text{mult-ARSS}}}$ emulates the behavior of the honest parties by invoking $\mathcal{F}_{\text{mult-RSS}}$ on their behalf. In the subsequent steps, the simulator proceeds by honestly following the remaining steps of the protocol.

Since all messages received by HP are either outputs from $\mathcal{F}_{\text{mult-RSS}}$ or derived from computations that are identically distributed in both the real and ideal worlds, the view of HP in the simulated execution is indistinguishable from its view in the real execution. Therefore, $\Pi_{\text{mult-ARSS}}$ is secure in the $\mathcal{F}_{\text{mult-RSS}}$ -hybrid model.

Simulator $\mathcal{S}_{\text{HP}}^{\Pi_{\text{Castor}}^{\text{Preproc}}}$

- $\mathcal{S}_{\text{HP}}^{\Pi_{\text{Castor}}^{\text{Preproc}}}$ follows the protocol on behalf of the parties in $\mathcal{P}$.
- For an input gate, consider P_d be the input provider and v be the input. It simulates in the following way:
 - It samples random shares of the mask using appropriate common keys such that P_d holds the random mask in clear. It generates replicated sharing of the mask $[\delta_v]$.
 - It simulates $\mathcal{F}_{\text{mult-RSS}}$ with inputs $([\Delta], [\delta_v])$.

- For an addition gate, it simulates by following the protocol steps.
- For an multiplication gate, consider x and y be the inputs and δ_x, δ_y be the corresponding masks. It simulates $\mathcal{F}_{\text{mult-ARSS}}$ with inputs $[\delta_x], [\delta_y]$.
- For an output, it generates $[\cdot]$ -sharing of a random mask δ_v using appropriate common keys.

Fig. 29: Simulator $\mathcal{S}_{\text{HP}}$ for $\Pi_{\text{Preproc}}^{\text{Castor}}$ for a semi-honest HP

Indistinguishability We now argue that the view of the semi-honest trusted party HP in the real execution of $\Pi_{\text{Preproc}}^{\text{Castor}}$ is indistinguishable from the view generated by the simulator $\mathcal{S}_{\text{HP}}^{\Pi_{\text{Preproc}}^{\text{Castor}}}$. The simulator interacts with HP by emulating the behavior of all parties in $\mathcal{P}$ and simulates each gate type in the following manner:

Input Gates: For each input provider P_d with input v , the simulator samples shares of a random mask δ_v using the appropriate common keys such that P_d holds the mask in the clear. The replicated sharing $[\delta_v]$ is generated honestly. The simulator then invokes the ideal functionality $\mathcal{F}_{\text{mult-RSS}}$ with inputs $([\Delta], [\delta_v])$ to simulate the masking process. In the real protocol, the same process occurs, and since the mask δ_v is uniformly random and independent of the actual input, the messages exchanged with HP are identically distributed.

Addition Gates: The simulator performs local computations exactly as specified in the protocol. Since no interaction with HP occurs beyond deterministic computation over masked shares, the real and simulated views are identical.

Multiplication Gates: Given masked inputs x and y with corresponding masks δ_x and δ_y , the simulator invokes the ideal functionality $\mathcal{F}_{\text{mult-ARSS}}$ using $[\delta_x]$ and $[\delta_y]$ as inputs. This step mimics the real protocol execution, where the honest parties would invoke $\Pi_{\text{mult-ARSS}}$ with these inputs. Since the functionality is simulated faithfully and HP is semi-honest, the resulting view remains indistinguishable.

Output Gates: The simulator generates a replicated $[\cdot]$ -sharing of a fresh, uniformly random output mask δ_v using the appropriate common keys. This step mirrors the behavior of the real parties in the protocol, and since the output mask is independent of the actual output, the view of HP remains unaffected.

In all cases, the messages received by HP in the simulated execution are either drawn from the same distributions as in the real execution or generated by invoking ideal functionalities that replace sub-protocols with their secure counterparts. Since HP is semi-honest and follows the protocol honestly, and the simulator replicates the exact message patterns and distributions, the view of HP in the real execution is computationally indistinguishable from its view in the simulated execution.

Hence, the simulation by $\mathcal{S}_{\text{HP}}^{\Pi_{\text{Preproc}}^{\text{Castor}}}$ is sound, and the preprocessing phase $\Pi_{\text{Preproc}}^{\text{Castor}}$ is secure against a semi-honest HP.

Simulator $\mathcal{S}_{\text{HP}}^{\Pi_{\text{Online}}^{\text{Castor}}}$

- *Preprocessing:* $\mathcal{S}_{\text{HP}}^{\Pi_{\text{Online}}^{\text{Castor}}}$ simulates the preprocessing phase by executing $\mathcal{S}_{\text{HP}}^{\Pi_{\text{Preproc}}^{\text{Castor}}}$ and prepare for the online phase.
- *Input gates:* $\mathcal{S}_{\text{HP}}^{\Pi_{\text{Online}}^{\text{Castor}}}$ sends randomly sampled m_v for all inputs v on behalf of each input provider. Then, it receives m_v for $n - t$ parties' inputs from HP.
- *Addition gates:* Since the addition gates are non-interactive, nothing to simulate.
- *Multiplication gates:* $\mathcal{S}_{\text{HP}}^{\Pi_{\text{Online}}^{\text{Castor}}}$ samples a random value $[m_z]_0$ and sends it to HP on behalf of all the parties. Then it receives a value m_z from HP.
- *Verification:*
 - $\mathcal{S}_{\text{HP}}^{\Pi_{\text{Online}}^{\text{Castor}}}$ simulates $\mathcal{F}_{\text{Rand}}$ and obtains ρ .
 - It computes $[\omega_{z_j}]_i = [t_{m_{z_j}}]_i - m_{z_j} \cdot [\Delta]_i$ for all $i \in [1, n]$ and $j \in [1, \ell]$.
 - It computes $[\omega_z]_i = \sum_{j=1}^{\ell} \rho_j \cdot [\omega_{z_j}]_i$.
 - It computes $[\omega_z]_0 = -\sum_{i=1}^n [\omega_z]_i$ and sends it to HP on behalf of each of the parties in $\mathcal{P}$.

- *Output:* $\mathcal{S}_{\text{HP}}^{\Pi_{\text{Online}}^{\text{Castor}}}$ receives the output from HP.

Fig. 30: Simulator $\mathcal{S}_{\text{HP}}$ for $\Pi_{\text{Online}}^{\text{Castor}}$ for a semi-honest HP

Indistinguishability We now argue that the view of the semi-honest HP in the real execution of $\Pi_{\text{Online}}^{\text{Castor}}$ is computationally indistinguishable from the view generated by the simulator $\mathcal{S}_{\text{HP}}^{\Pi_{\text{Online}}^{\text{Castor}}}$. The simulation proceeds as follows:

Preprocessing Phase: The simulator first invokes $\mathcal{S}_{\text{HP}}^{\Pi_{\text{Preproc}}}$ to simulate the preprocessing phase. From the previous indistinguishability argument for $\mathcal{S}_{\text{HP}}^{\Pi_{\text{Preproc}}}$, the view of HP at the end of this phase is computationally indistinguishable from its view in the real execution.

Input Gates: For each input value v provided by a party in $\mathcal{P}$, the simulator sends a randomly sampled masked value m_v to HP. In the real protocol, these masks are generated using shares and preprocessing, but the mask hides the true input due to information-theoretic security. Thus, the distribution of the masked inputs is independent of the actual inputs and identically distributed in both the real and simulated executions. Receiving m_v for the remaining inputs from HP is consistent with the real protocol and does ensure the indistinguishability.

Addition Gates: Addition gates are evaluated locally and involve no interaction with HP. Hence, there is nothing to simulate and the views trivially match.

Multiplication Gates: The simulator samples a fresh random value $[m_z]_0$ and sends it to HP as the additive share of the masked product. In the real protocol, this value is a masked version of the multiplication output, with masking done using values generated in preprocessing. Due to the use of a fresh mask and the simulation of preprocessing, the distribution of these values remains identical across both real and simulated executions. The value m_z received from HP is used in subsequent steps and does not affect the indistinguishability.

Verification Step: The simulator samples the verification challenge ρ by simulating $\mathcal{F}_{\text{Rand}}$.

It then computes the linear combination ω_z from tag values and the verification key Δ . Since these values are derived from previously simulated (or randomly sampled) quantities, and all computations are local linear operations, the resulting messages sent to HP are distributed identically in both the real and simulated executions.

In particular, the computation of $[\omega_z]_0$ ensures that the overall sharing of ω_z sums to zero, as required by the protocol. This mirrors the real verification step exactly. Furthermore, $\mathcal{A}$ can pass the verification with incorrect reconstructed value with a probability $O(\frac{1}{\mathbb{F}})$ (follows the similar argument provided in F.1 and [29]).

Output Step: Finally, the simulator receives the output from HP, consistent with the real protocol. As output reconstruction is a local operation and no secret values are revealed to HP, this step does not impact indistinguishability.

Conclusion: Each message generated by the simulator either matches the distribution of the corresponding real message due to the information-theoretic hiding of masks, or results from computations on previously simulated values that are indistinguishable from the real ones. Therefore, the overall view of HP in the simulated execution of $\Pi_{\text{Online}}^{\text{Castor}}$ is computationally indistinguishable from its view in the real execution.

Hence, we establish that $\Pi_{\text{Online}}^{\text{Castor}}$ is secure against either a malicious $\mathcal{A}$ corrupting all but one parties in $\mathcal{P}$ or a semi-honest HP in the $\Pi_{\text{Preproc}}^{\text{Castor}}$ -hybrid model.